

Self-Tallying e-Voting using Homomorphic Time-Lock Puzzles and zk-SNARKs

Muhammad ElSheikh, Amr M. Youssef, *Senior Member, IEEE*, M. Anwar Hasan, *Senior Member, IEEE*

Abstract—Achieving fairness, verifiability, and abandon resistance poses challenges within e-voting protocols. This paper introduces a privacy-preserving self-tallying e-voting system leveraging blockchain technology. The system supports diverse e-voting models, including ‘Yes/No’, approval voting with multiple candidates, and score voting. By employing linearly homomorphic time-lock puzzles (LHTLPs) along with verifiable delay functions (VDFs) and zero-knowledge Succinct Non-interactive Argument of Knowledge schemes (zk-SNARKs), the proposed system ensures crucial security properties, including voter anonymity and eligibility, as well as ballot privacy and validity. It also provides efficient individual and universal verifiability (end-to-end verifiability), and dispute-freeness. More importantly, the system demonstrates fairness and abandon resistance. Furthermore, the evaluation of the proof-of-concept implementation on the Ethereum blockchain indicates that on-chain gas costs are either fixed or increasing slowly and logarithmically with the number of voters.

Index Terms—E-voting, self-tallying, blockchain, zk-SNARK, time-lock puzzles, VDF, privacy, verifiability.

I. INTRODUCTION

Electronic Voting (E-voting) is a system that enables voters to cast their votes electronically. E-voting, compared to traditional paper-based voting, promises (i) long-term cost-effectiveness by eliminating the expense of printing and distributing ballots, (ii) quick, transparent, and accurate tallying by removing human factors, (iii) accessibility and convenience, allowing voters to cast their ballots at any time from any location, and (iv) support a large number of voters. With the increasing popularity of e-voting, concerns arise regarding the security and integrity of the voting process. Generally, e-voting systems must satisfy security properties such as voters’ anonymity, privacy, and eligibility, as well as ballots privacy and validity. Additionally, they must address fairness, transparency, and the accuracy of tallying [1], [2]. Furthermore, e-voting systems must meet logistical properties such as efficiency and accessibility.

Early forms of e-voting, such as Internet voting, heavily rely on trusted central authorities. This reliance raises concerns about voters’ privacy, transparency, tamper-proofing of voting records, and the tallying process, in addition to creating a single point of failure. Therefore, several proposed protocols utilize cryptographic algorithms such as secret sharing [3], ring signatures [4], blind signatures [5], and multi-party computation (MPC) [6] to distribute trust among non-colliding

authorities. However, verifiability of casting and tallying the ballots remains an issue [7].

Using blockchain technology as a decentralized append-only ledger and utilizing smart contracts as trusted execution environments have prompted the development of numerous protocols across several fields [8], [9], including e-voting aimed at addressing various challenges [10].

Despite these advancements, two significant issues persist: the challenge of tallying without relying on trusted authorities and determining the number of voters that can be effectively handled. Due to inherent blockchain limitations, such as block mining rates and gas limits, blockchain-based e-voting protocols encounter scalability challenges. As the number of voters increases, the throughput of verified votes per second is likely to decrease because the cost of on-chain computation per vote rises, potentially at a linear rate. Furthermore, beyond a certain point, this rising cost may limit the total number of participating voters.

Self-tallying e-voting protocols [11], [12] are interactive protocols executed among a group of voters without relying on a trusted authority. In such protocols, any observer can tally the result at the end without the need for a trusted entity. The initial self-tallying protocols introduced by Kiayias and Yung [11] and Groth *et al.* [12] are suitable for small-scale boardroom voting but necessitate a significant number of rounds. Hao *et al.* [13] proposed the Open Vote Network (OVN), a two-round self-tallying protocol that achieves better efficiency in terms of the number of rounds while maintaining the security properties of Groth *et al.* [12]. However, OVN faces challenges as its second round involves computations that increase linearly with the number of voters. It also encounters abortive and adaptive issues, meaning tallying cannot proceed if at least one voter refuses to cast the ballot (lack of abandon resistance). Additionally, the last voter has an advantage of partially tallying the result before casting his ballot, potentially leading to vote changes or aborting based on this partial tallying (lack of fairness). Khader *et al.* [14] partially improved OVN’s fairness by preventing the last voter from changing the vote through vote commitment in the first round. They also introduced a third round, a recovery round, wherein remaining voters collaborate to tally the result if some voters refuse to cast their ballots. Javani and Sherman proposed BVOT [15], a self-tallying e-voting protocol employing multiparty threshold homomorphic encryption and oblivious transfer to achieve fairness among voters. Nevertheless, BVOT lacks abandon resistance.

McCorry *et al.* [16] were the first to attempt implementing OVN on the Ethereum blockchain. However, their implementation suffers from the same issues as OVN (lack of abandon

M. ElSheikh and M. A. Hasan are with the Department of Electrical and Computer Engineering, University of Waterloo, Canada. M. ElSheikh is also with National Institute of Standards (NIS), Cairo, Egypt
E-mail: {mhgaels, ahasan}@uwaterloo.ca

A. Youssef is with the Concordia Institute for Information Systems Engineering, Concordia University, QC, Canada.

resistance and fairness), along with concerns due to heavy on-chain computations. In particular, the on-chain computation per voter—and thereby its gas consumption—increases linearly with the number of voters. More importantly, this computation has to be completed within a single transaction. Consequently, this imposes a limit on the number of participating voters, given that blockchains impose a gas consumption limit per block. Seifelnasr *et al.* [17], and ElSheikh and Youssef [18] partially addressed the heavy on-chain computation issue by delegating computations to a trustless administrator in a verifiable manner. Li *et al.* [19] and Yang *et al.* [20] implemented the approach introduced by Khader *et al.* [14] to enhance fairness in OVN.

A time-lock puzzle [21] is a cryptographic construction that conceals information for a predetermined period, *i.e.*, it acts as a delay mechanism to prevent immediate access. Li *et al.* [22] utilized linearly homomorphic time-lock puzzles (LHTLPs) [23] to construct a self-tallying e-voting system with multiple candidates in which each voter generates independent LHTLPs, a puzzle for each candidate. This approach prevents any criteria for candidate selection and causes issues like time-consuming tallying, expensive on-chain verifications, and potential threats to ballot privacy. Additionally, validating the result requires re-tallying. Furthermore, preventing double voting demands on-chain comparisons potentially causing a linearly increasing cost of on-chain computation.

Regarding the security of e-voting systems, we reference several security incidents in previous systems. In Internet e-voting, the iVote¹ system used in New South Wales, Australia, in 2015 had vulnerabilities that could lead to manipulations or capture of private information [24]. Similarly, the Estonian internet e-voting system was found vulnerable to attacks that could alter election results by compromising election servers or voters' devices [25]. For blockchain-based e-voting, the Voatz² application, used in the 2018 US midterm elections, was reported to have serious security vulnerabilities that allowed attackers to monitor votes and manipulate ballots [26]. The blockchain-based voting system used in Moscow's city council elections in 2019 also faced significant security issues, with Gaudry and Golovnev [27] identifying two attacks stemming from encryption implementation mistakes. These incidents underscore the need for continuous improvement in the security of e-voting systems.

Motivations. Our motivation for designing a secure e-voting protocol is to overcome the limitations of paper-based voting and, at the same time, achieve long-term cost-effectiveness, quick, transparent, and accurate tallying, accessibility convenience, and support a large number of voters. Furthermore, e-voting systems are increasingly recognized as essential in various scenarios. For instance, the COVID-19 pandemic has made traditional paper-based voting potentially hazardous to health and occasionally impractical within many organizations due to remote work setups. Additionally, the rise of decentralized autonomous organizations (DAOs) highlights the critical need for robust and secure blockchain-based e-voting systems.

These systems can facilitate decentralized decision-making processes while ensuring transparency, integrity, and fairness. While existing self-tallying e-voting protocols, such as OVN, though making strides in addressing some challenges, still exhibit significant limitations, including a linearly increasing cost of on-chain computations and a lack of abandon resistance and fairness, as illustrated previously.

Contribution. This paper introduces a privacy-preserving self-tallying e-voting system built on blockchains. Our system supports these e-voting models: 'Yes/No', *approval voting* with multiple candidates, and *score voting*. In addition to the essential security properties such as voters' anonymity, privacy, and eligibility, as well as ballot privacy and validity, the system offers dispute-freeness and efficient end-to-end verifiability by combining individual and universal verifiability. More importantly, it achieves fairness, abandon resistance, and reasonable scalability in terms of the execution time of off-chain computation and the cost of on-chain computation, and ensures a robust level of availability and resistance against Denial-of-Service (DoS) attacks. Our contribution can be summarized as follows.

- We firstly present a self-tallying e-voting protocol for the 'Yes/No' e-voting model. In the protocol, voters employ LHTLPs and zero-knowledge Succinct Non-interactive Argument of Knowledge schemes (zk-SNARKs) to encrypt their votes and post them anonymously on the blockchain. Subsequently, any observer can leverage the linear homomorphism property of LHTLP to tally the result by solving the accumulated LHTLP.
- We employ verifiable delay functions (VDFs) to introduce a protocol named $\prod_{LHTLP}$, which enables a solver of LHTLPs to efficiently demonstrate the accuracy of their solution to a given puzzle. As the e-voting tallying result is encrypted as an accumulated LHTLP, an e-voting observer can offer an efficient proof of the correctness of the tallying result using $\prod_{LHTLP}$.
- We demonstrate the protocol's efficiency in supporting two non 'Yes/No' e-voting models, namely, approval voting with multiple candidates and score voting. Furthermore, the protocol can facilitate multiple e-voting events with the same setup and registration phases.
- We develop a proof-of-concept implementation³ on the Ethereum blockchain to evaluate the functionality and performance. The evaluation indicates that the on-chain gas costs essentially remain fixed or increase slowly and logarithmically with the change in the number of voters.

II. PRELIMINARIES

Let N be a strong RSA integer, *i.e.*, $N = P \cdot Q$ where P and Q are random safe primes s.t. $P = 2p' + 1$ and $Q = 2q' + 1$, where p' and q' are also primes. Let $\mathbb{J}_N$ be the group of elements of $\mathbb{Z}_N^*$ with Jacobi symbol $+1$. Let $\mathbb{QR}_N$ denote the quadratic residues group of elements where $\mathbb{QR}_N \triangleq \{z^2 \bmod N : z \in \mathbb{Z}_N^*\}$ and $\mathbb{QR}_N \subset \mathbb{J}_N$. Since N is a strong RSA integer, $\mathbb{J}_N$ and $\mathbb{QR}_N$ are cyclic groups with order $\phi(N)/2$ and $\phi(N)/4$, respectively, where ϕ is the Euler totient function.

¹<https://elections.nsw.gov.au/about-us/reports/ivote-reports>

²<https://voatz.com/>

³Github: <https://github.com/mhgharieb/zk-Evote-HTLP>

On common input (N, x, T, y)

- 1) If $T = 1$ then $\mathcal{V}$ outputs **accept** if $y = x^{2^T} = x^2 \pmod{N}$ and **reject** otherwise.
If $T \geq 2$ go to the next step.
- 2) $\mathcal{P}$ sends $\mu' = x^{2^{(T/2)-1}} \pmod{N}$ to $\mathcal{V}$.
- 3) If $\mu' \notin \mathbb{Z}_N^*$, $\mathcal{V}$ outputs **reject** and stops, otherwise $\mathcal{V}$ computes $\mu = \mu'^2 \pmod{N}$ (note that $\mu \in \mathbb{QR}_N$).
- 4) $\mathcal{V}$ samples at random $\alpha \xleftarrow{\$} \mathbb{Z}_{2^\lambda}$ and sends it to $\mathcal{P}$.
- 5) $\mathcal{P}$ and $\mathcal{V}$ output $(N, x', T/2, y')$ where:

$$\begin{aligned} x' &= x^\alpha \cdot \mu \left(= x^{\alpha+2^{T/2}} \right) \\ y' &= \mu^\alpha \cdot y \left(= x^{\alpha \cdot 2^{T/2} + 2^T} = x^{2^{T/2}} \right) \end{aligned}$$

Fig. 1: The halving subprotocol [29]

Let H and H_λ be two collision-resistant hash functions s.t. $H : \mathbb{F}_p \times \mathbb{F}_p \rightarrow \mathbb{F}_p$, and $H_\lambda : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, and $H_\lambda(a, b) = H_\lambda(a || b)$, $||$ is the concatenations of bit-strings.

A. Pietrzak's Verifiable Delay Function

A verifiable delay function (VDF) [28] is a function that maps an input $x \in \mathcal{X}$ to a unique output $y \in \mathcal{Y}$ in a fixed amount of time, even when this function run on a parallel computer. More importantly, anyone can efficiently verifies the correctness of the output y . Recently, Pietrzak [29] and Wesolowski [30] proposed two VDF constructions that rely on Rivest, Shamir and Wagner (RSW) time-lock puzzle [21]. In this work, we use the VDF due to Pietrzak [29] since its proof generation is more efficient [31]. Moreover, we focus on its version where $x, y \in \mathbb{QR}_N$ because its verification algorithm can be efficiently implemented over blockchains such as Ethereum as we will propose in this work.

Pietrzak's VDF protocol Π_{VDF} involves two parties, a prover $\mathcal{P}$ and a verifier $\mathcal{V}$, who have an RSW puzzle (N, x, T) and a statistical security parameter λ , where N is an RSA integer, $x \in \mathbb{QR}_N$, and $T \in \mathbb{N}$ is a time parameter s.t. $T = 2^t$ for simplicity. Parties $\mathcal{P}$ and $\mathcal{V}$ do not know the factorization of N . Then, $\mathcal{P}$ solves that RSW puzzle and tries to convince $\mathcal{V}$ of the correct solution as follows:

Evaluation: $\mathcal{P}$ computes $y = x^{2^T} \pmod{N}$ by performing T sequential squarings and sends the output y to $\mathcal{V}$.

Verification: If $y \notin \mathbb{QR}_N$, $\mathcal{V}$ outputs **reject** and stop, otherwise $\mathcal{P}$ and $\mathcal{V}$ participate in the recursive halving subprotocol (Fig. 1) for the tuple (N, x, T, y) as the input. For each round, the halving subprotocol's output is either $(N, x', T/2, y')$ as computed in the 5th step (see Fig. 1), which is used as input for the next round, or a final output of either **reject** or **accept** indicating whether $\mathcal{P}$ has honestly solved the puzzle correctly.

The protocol can be transferred to *non-interactive* one by applying the Fiat-Shamir heuristic [32]. Concretely, the prover hashes the input of each round and uses it as the challenge of that round, i.e., for i -th round of the protocol, $\alpha_i = H_\lambda(T_i = T/2^i, x_i, y_i, \mu_i)$. Hence, the overall proof π_{VDF} of the verification will be $\{\mu'_i\}_{i \in [t]}$.

- $pp \leftarrow \text{HP.PSetup}(1^\tau, T)$: Output $pp := (T, N, g_J, h)$, where N is a strong RSA integer, g_J is a random generator of $\mathbb{J}_N$, and $h := g_J^{2^T} \pmod{N}$.
- $Z \leftarrow \text{HP.PGen}(pp, d)$: Output $Z := (u, v)$, where $u := g_J^r \pmod{N}$ and $v := h^{r \cdot N} \cdot (1 + N)^d \pmod{N^2}$ s.t. $r \xleftarrow{\$} \{1, \dots, N^2\}$.
- $d \leftarrow \text{HP.PSolve}(pp, Z)$: Output $d := \frac{(v/w^N \pmod{N^2})-1}{N}$, where $w := u^{2^T} \pmod{N}$ is computed by repeated squaring.
- $Z' \leftarrow \text{HP.PEval}(+, pp, Z_1, \dots, Z_n)$: Output $Z' = (\hat{u}, \hat{v})$, where $\hat{u} := \prod_{i=1}^n u_i \pmod{N}$ and $\hat{v} := \prod_{i=1}^n v_i \pmod{N^2}$.

Fig. 2: Linearly HTLP (LHTLP) Algorithms [23]

B. Linearly Homomorphic Puzzles

Homomorphic Time-Lock Puzzles (HTLP) [23] extend the concept of time-lock puzzles. It enables the homomorphic combination of a set of input puzzles to yield an output puzzle. The solution of this output puzzle corresponds to the evaluation of a function over the solutions of the input puzzles, without requiring knowledge of those solutions. A homomorphic time-lock puzzle (HTLP) scheme is composed of four algorithms defined as follows:

- $pp \leftarrow \text{HP.PSetup}(1^\tau, T)$: a probabilistic algorithm that takes a security parameter τ and a time hardness parameter T as input, and outputs public parameters pp .
- $Z \leftarrow \text{HP.PGen}(pp, d)$: a probabilistic algorithm that takes pp and a solution d as input, and outputs a puzzle Z .
- $d \leftarrow \text{HP.PSolve}(pp, Z)$: a deterministic algorithm that takes pp and Z as input, and outputs a solution d .
- $Z' \leftarrow \text{HP.PEval}(f, pp, Z_1, \dots, Z_n)$: a probabilistic algorithm that takes pp , a function f , and a set of n puzzles $(Z_1, \dots, Z_n)$ as input, and outputs a puzzle Z' .

Malavolta and Thyagarajan [23] proposed two HTLP schemes: a linearly HTLP and a multiplicatively HTLP. Our proposed e-voting protocol utilizes the linearly HTLP (LHTLP) over $(\mathbb{Z}_N, +)$ as illustrated in Fig. 2.

C. Merkle Tree and Incremental Merkle Tree Algorithm

A Merkle tree is a perfect binary tree that utilizes a secure cryptographic hash function to organize data items into leaf nodes. Each leaf stores the hash of a data item and non-leaf nodes store hashes of their two child nodes until reaching a single root hash (the Merkle root).

A partial Merkle tree filled up to k is an append-only Merkle tree where the leftmost k leaves are filled with hashes of data items while the remaining leaves are filled with hashes of zeros. The incremental Merkle tree algorithm takes a partial Merkle tree filled up to index k and a new data item. Next, it efficiently appends the hash of this data item into the $(k+1)^{th}$ leaf and produces a new partial Merkle tree filled up to $k+1$ with a new Merkle root [33]. To efficiently do this, the algorithm keeps track of the Merkle proof for the next empty leaf. Fig. 3 illustrates how the incremental Merkle tree algorithm updates the left tree to the right tree.

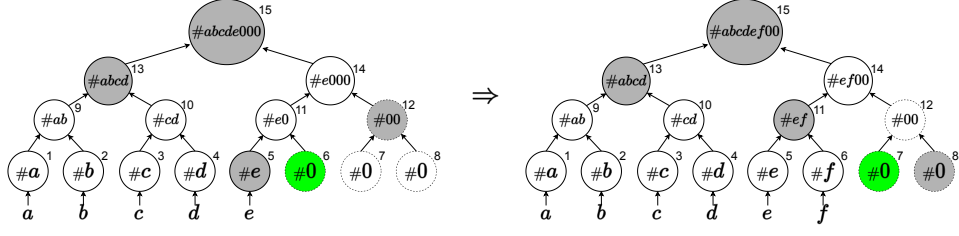


Fig. 3: Illustration of the incremental Merkle tree algorithm, where the left tree is updated to the right tree by appending (f) to the 6th node. The # symbol denotes the hash value. The green node is the next empty node and the gray ones are its Merkle proof.

D. zk-SNARKs

zk-SNARKs or Zero-knowledge Succinct Non-interactive Argument of Knowledge schemes are cryptographic schemes that allow a prover to convince a verifier of the truth of a statement without any prior interaction between them [34].

Consider an arithmetic circuit C which has an associated relation $\mathcal{R}_C$ and a language $\mathcal{L}_C$. Given an input statement $\vec{s}$ and a corresponding witness $\vec{w}$ s.t. $(\vec{s}, \vec{w}) \in \mathcal{R}_C$, a zk-SNARK for proving the satisfiability of this arithmetic circuit involves three polynomial-time algorithms [34], [35]:

- $(pk, vk) \leftarrow \text{Setup}(1^\kappa, C)$: it takes a security parameter κ and generates two keys; a proving key (pk) and a verifying key (vk) that serve as public parameters for C .
- $\pi \leftarrow \text{Prove}(pk, \vec{s}, \vec{w})$: it takes as input pk and $(\vec{s}, \vec{w}) \in \mathcal{R}_C$, and constructs a non-interactive proof π for the statement $\vec{s} \in \mathcal{L}_C$ without revealing the witness.
- $0/1 \leftarrow \text{Verify}(vk, \vec{s}, \pi)$: given vk , it outputs 1 if the proof π is valid for the statement $\vec{s} \in \mathcal{L}_C$, and 0 otherwise.

zk-SNARKs offer the following security properties [34], [35]:

- 1) **Perfect Completeness:** For any $(\vec{s}, \vec{w}) \in \mathcal{R}_C$, an honest prover always successfully convinces a verifier, i.e., the Verify algorithm returns 1 with a probability 1.
- 2) **Knowledge Soundness:** If the verifier accepts a proof by a PPT adversary, the witness can be computed by a PPT extractor EXT that gets full access to the adversary's state.
- 3) **Succinctness:** The size of the honestly-generated proof is polynomial in the security parameter κ , and the Verify algorithm runs in polynomial time relative to $\kappa + |\vec{s}|$.
- 4) **Prefect Zero-Knowledge:** This property guarantees that an honestly-generated proof leaks no information about the witness. Formally, there is a polynomial-time simulator Sim s.t. $\forall \kappa, \forall (\vec{s}, \vec{w}) \in \mathcal{R}_C$ and for all adversaries $\mathcal{A}$

$$\Pr \left[\mathcal{A}(\pi) = 1 \mid \begin{array}{l} (pk, vk) \leftarrow \text{Setup}(1^\kappa, C) \\ \pi \leftarrow \text{Prove}(pk, \vec{s}, \vec{w}) \end{array} \right] =$$

$$\Pr \left[\mathcal{A}(\pi) = 1 \mid \begin{array}{l} (pk, vk, \text{trp}) \leftarrow \text{Sim}(1^\kappa, C) \\ \pi \leftarrow \text{Sim}(\vec{s}, \text{trp}) \end{array} \right]$$

III. PROTOCOL OVERVIEW

In this section, we present an overview of the proposed blockchain-based e-voting protocol, including its participants, phases, and the goals it aims to achieve. Additionally, we outline the threat model.

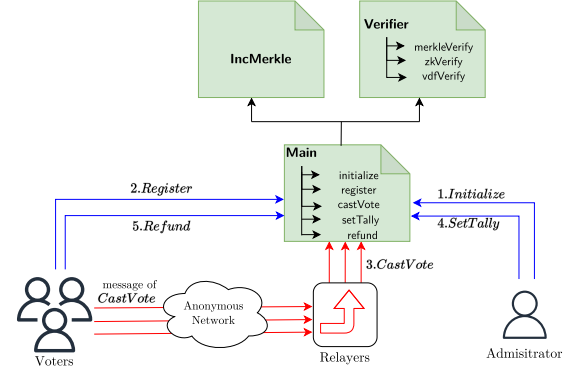


Fig. 4: Overview of the proposed system where green components are the smart contracts. Voters and Administrator directly send transactions (blue), while voters send transactions anonymously via relayers (red). Black arrows are calls between smart contracts.

A. Protocol Participants

The protocol involves interactions between an election administrator, a group of voters (V_j), trustless relayers, and the smart contracts on the blockchain, as illustrated in Fig. 4.

Election Administrator: This role can be played by one of the voters or a third party for deploying the smart contract and performing heavy computations during the protocol, such as tallying the voting results, in a verifiable manner.

Voters: They are individuals or parties eligible to participate in the voting event. Voters are uniquely identified by their addresses on the blockchain and must register these addresses to become legitimate voters and cast their encrypted ballots.

Trustless Relayers: Relayers are blockchain accounts that offer to send and pay for transactions on behalf of others. In return, they receive rewards from the smart contract.

Smart Contracts: Their functionality includes collecting encrypted ballots, verifying ballot validity, enforcing the rules of the election, and verifying the tallying result. The system consists of a frontend smart contract Main along with two backend smart contracts:

- 1) **Main:** It comprises five functions, namely: initialize, register, castVote, setTally, and refund. These functions receive transactions from other participants in the protocol and handle the flow of the protocol phases.
- 2) **IncMerkle:** It efficiently implements the incremental Merkle tree algorithm.
- 3) **Verifier:** It contains three functions, namely: merkleVerify, zkVerify, and vdfVerify. These functions are responsible

for verifying the proofs of Merkle tree membership, zk-SNARK, and VDF, respectively.

B. Protocol Phases

The protocol comprises five chronologically ordered phases: Setup, Registration, Encrypted vote casting, Result tallying, and finally Refund.

Setup phase: The voters, alongside the administrator, participate in two MPC-based ceremonies to select the protocol's parameters and to generate the keys for a zk-SNARK circuit.

Registration phase: Each voter chooses a secret value and then commits it to the smart contract, which appends the voters' commitments within an incremental Merkle tree. Also, the smart contract collects a collateral fee from each voter to create a dedicated fund for rewarding the trustless relayers.

Encrypted vote casting phase: Voters encrypt their votes by converting them into LHTLPs. LHTLPs enable each voter to create a ballot independently of other voters while maintaining ballot privacy. The voter also generates a zk-SNARK proof to demonstrate being a registered voter and that the encrypted ballot is properly formatted. These zk-SNARKs facilitate efficient verification of the ballots on-chain. Subsequently, the voter sends the encrypted ballot to one of the trustless relayers through an anonymous network as illustrated in Fig. 4. The relayer then forwards the ballot to the smart contract and pays the transaction fee on behalf of the voter. In response, the smart contract firstly verifies the zk-SNARK proofs, and then accumulates the LHTLPs in a single puzzle, and finally rewards the relayer. As a result, voters maintain their anonymity by breaking the link between their addresses and their ballots. Any party can trace the transaction fee back to the relayer and the rewards back to the smart contract fund. However, while it is possible to associate transactions with the group of registered voters, it remains impossible to identify specific voters.

Result tallying phase: At the end of casting votes, the administrator retrieves the accumulated puzzle and solves it. The resulting solution is the final tally. During solving the puzzle, the administrator efficiently extracts a VDF proof that validates the solution's correctness. Consequently, the smart contract verifies the correctness of the final tally.

Refund phase: After posting the result, the smart contract divides the remaining fund among the registered voters.

C. Threat Model

In our threat model, we assume that the cryptographic primitives are secure. Furthermore, we assume that the blockchain is tamper-resistant and the execution of the smart contract is trusted, *i.e.*, adversaries can not control over more than 50% of the computation power of the blockchain mining/validation nodes. Additionally, we require the participation of at least one honest voter in the MPC-based ceremonies during the setup phase. We exclude some threats that extend beyond the scope of our e-voting system, such as device hacking/exploitation and phishing/social engineering attacks targeting the protocol participants.

We identify various potential adversaries that could compromise the integrity of an e-voting system as follows:

- Adversary ($\mathcal{A}$):
 - 1) Registration phase: Impersonate legitimate voters.
 - 2) Encrypted vote casting phase: Forge or invalidate valid ballots, or compromise voter privacy by linking voters to their ballots.
 - 3) Result tallying phase: Manipulate election results.
- Election administrator:
 - 1) Setup phase: Attempt to manipulate the protocol security parameters during the MPC-based ceremonies, allowing the manipulation of the remaining phases.
 - 2) Encrypted vote casting phase: The forging of ballots and early access to plaintext votes.
 - 3) Result tallying phase: Selectively discount valid votes and alter the final tally.
- Voters:
 - 1) Setup phase: Attempt to manipulate the protocol security parameters during the MPC-based ceremonies, allowing the manipulation of the remaining phases.
 - 2) Registration phase: Register multiple times, impersonate legitimate voters by mimicking voting behavior.
 - 3) Encrypted vote casting phase: Cast a ballot multiple times, submit forged or invalid ballots, disrupt the election process by abstaining from participation.
- Trustless relayers:
 - 1) Encrypted vote casting phase: Engage in ballot tampering by modifying encrypted ballots before relaying them to the smart contract, conduct denial of service (DoS) attacks against specific voters or candidates by selectively disclosing or suppressing transactions, and attempt to link encrypted ballots to specific voters' addresses.

Remark 1. Our proposed system requires two MPC-based ceremonies during the setup phase: one for the zk-SNARK trusted setup and another for the RSA modulus parameter setup. These two ceremonies operate under a dishonest majority model.

While zk-SNARK ceremonies typically run in a round-robin fashion—with each participant passing their contribution to the next—this structure could allow malicious parties to exclude honest ones. To address this, a trusted coordinator can be employed (for instance, using a public bulletin board such as a blockchain, as proposed in [36]) to collect and forward contributions, ensuring that no single party or colluding group can control the process.

We recommend using [37] for the RSA modulus setup despite its lack of guaranteed output delivery because it is lightweight, scalable, and achieves public verifiability. More importantly, it features an identifiable-abort property that allows for the identification of dishonest parties when the protocol aborts, thereby facilitating accountability for dishonest parties. Nevertheless, if guaranteed output delivery is the primary concern, the ceremony can be performed using less efficient general-purpose MPC with an honest majority, such as those described in [38], [39].

It should also be noted that the distributed ceremonies follow synchronous communication models. Furthermore, voters and the administrator operate within deterministic time windows during the e-voting protocol. In other words, the protocol phases adhere to strict deadlines tied to blockchain block heights, and transactions sent by both voters and the administrator must be submitted within these windows to be considered valid.

D. Design Goals

The system has to achieve the following security properties:

- Anonymity (Unlinkability): Ballots must be unlinkable to the voters who cast them, and it should be impossible for an adversary to determine if a registered voter has voted.
- Time-bounded ballot privacy [22]: The votes on ballots must remain confidential at least for a predefined time period that covers the casting vote phase.
- Ballot validity: All anonymous ballots submitted to the blockchain must adhere to proper formatting, be cast by eligible registered voters, and remain free from tampering.
- Dispute-freeness [11]: The protocol must proactively prevent participant misbehavior at any phase to eliminate potential disputes or disagreements among them.
- Fairness [11]: No party should have the ability to partially tally the result before the casting vote phase concludes.
- Double voting resistance: Eligible voters can neither register multiple times nor cast multiple ballots.
- Individual verifiability [40]: Every voter is able to confirm the integrity of the encrypted ballot, *i.e.*, the encrypted ballot is *cast as intended* and *recorded as cast*.
- Universal verifiability [11]: It should be easy to convince any interested party that all valid votes have been included in the final tally, *i.e.*, *counted as recorded*. It ensures that no one can falsify the final tally⁴.
- Abandon resistance: If any entity, including the administrator, voters, or candidates, engages in the protocol but aborts before the end of the protocol, the final tally is still be efficiently computed.

Also, the protocol should possess the following features:

- Self-Tallying: Interested parties should be capable of tallying the results themselves and persuading others of the accuracy of their calculations.
- Scalability: As the number of voters increases, the cost of on-chain computation per vote rises, potentially at a linear rate. As a result, the throughput of verified votes per second is likely to decrease. Furthermore, beyond a certain point, this rising cost may limit the total number of participating voters. Limiting this cost growth can enhance overall throughput. Throughout this work, a blockchain-based e-voting is considered ‘scalable’ when both the execution time of off-chain computation, conducted by each protocol participant, and the cost of on-chain computation, paid by each protocol participant, either remain constant or increase logarithmically with the number of voters.

⁴End-to-End verifiability [7] is achieved by combining individual verifiability and universal verifiability.

- 1) $\mathcal{P}$ sends $w := u^{2^T} \bmod N$ to $\mathcal{V}$.
- 2) $\mathcal{P}$ and $\mathcal{V}$ participate in Π_{VDF} for the tuple (N, u, T, w) .
- 3) $\mathcal{V}$ outputs **accept** if the three statements are true:
 - a) The argument (N, u, T, w) of Π_{VDF} is accepted,
 - b) $d < N$,
 - c) $w^N \cdot (1 + N)^d \bmod N^2 = v$.
 Otherwise, $\mathcal{V}$ outputs **reject**.

Fig. 5: The protocol Π_{LHTLP}

IV. PROTOCOL DESIGN

Our proposed protocol integrates two fundamental components: the LHTLP [23] and Pietrzak’s VDF [29]. In our protocol, each voter encrypts a vote as a LHTLP. By accumulating individual puzzles, the encrypted tallying result are available in the form of an accumulated LHTLP, leveraging the linear homomorphism property of LHTLPs. Subsequently, the administrator obtains the plaintext tallying result by solving that accumulated LHTLP. Finally, the smart contract validates the final tally by utilizing Pietrzak’s VDF scheme to verify the solution of the accumulated LHTLP.

A. Verifiability for the solution of a LHTLP

The output of the LHTLP $Z = (u, v)$ lies in $\mathbb{J}_N \times \mathbb{Z}_{N^2}^*$, while Pietrzak’s VDF operates on $\mathbb{QR}_N$. Given that $\mathbb{QR}_N \subset \mathbb{J}_N$, we adapt the LHTLP, as illustrated in Fig. 2, s.t. $(u, v) \in \mathbb{QR}_N \times \mathbb{Z}_{N^2}^*$. This adaptation facilitates verification using Pietrzak’s VDF. During HP.PSetup, the protocol picks g randomly as a generator of $\mathbb{QR}_N$ instead of $\mathbb{J}_N$.

Soundness of Pietrzak’s VDF protocol (Π_{VDF}). Recalling the proof of soundness for Pietrzak’s VDF protocol by Boneh *et al.* [41], Pietrzak’s VDF protocol is a succinct public-coin interactive argument for the language

$$\mathcal{L}_{VDF} = \{(\mathbb{G}, x, T, y) : y = x^{2^T} \text{ in } \mathbb{G}\}$$

where $x, y \in \mathbb{G}$ and $\mathbb{G}$ is a finite abelian group of unknown order in which the low order assumption holds, *i.e.*, it is hard to find a low order element in $\mathbb{G}$.

Theorem 1 ([41, Theorem 1]). *Suppose the low order assumption holds for $\mathbb{G}$. Then Pietrzak’s succinct argument has negligible soundness error.*

Recall that N is a strong RSA integer and its factorization is unknown. Consequently, its quadratic residues group $\mathbb{QR}_N$ contains no elements of low order other than 1, *i.e.*, the low order assumption holds unconditionally in this group. By relying on the low order assumption, the soundness of Pietrzak’s VDF protocol is proven for all $x \in \mathbb{QR}_N$ even when x is not a generator of $\mathbb{QR}_N$. **Correctness of the solution for a LHTLP.** Now, we propose a protocol, denoted by Π_{LHTLP} , that allows a puzzle solver ($\mathcal{P}$) to prove to a verifier ($\mathcal{V}$) that the solution d driven from solving a puzzle $Z = (u, v) \in \mathbb{QR}_N \times \mathbb{Z}_{N^2}^*$ is correct. The protocol (Π_{LHTLP}) is a succinct public-coin interactive argument for the language

$$\mathcal{L}_{LHTLP} = \{(N, T, u, v, d) : v = (u^{2^T})^N \cdot (1 + N)^d \bmod N^2\}$$

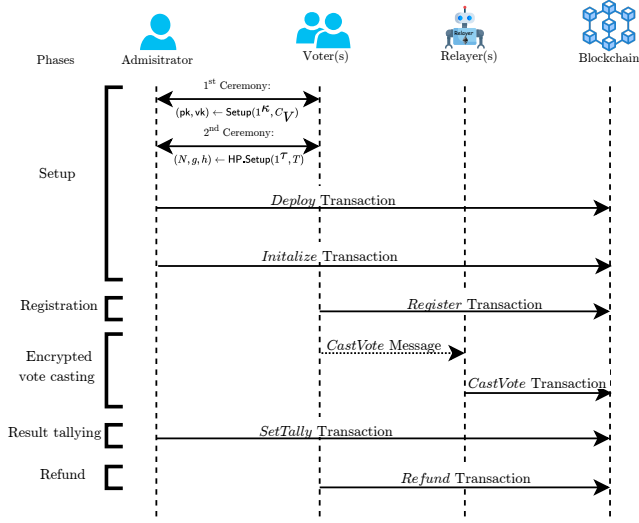


Fig. 6: Interactions between the protocol's participants, where voters anonymously send *CastVote* messages to trustless relayers.

where N is a strong RSA integer and $u \in \mathbb{QR}_N$. $\prod_{LHTLP}$ is illustrated in Fig. 5.

Theorem 2. Suppose N is a strong RSA integer with an unknown factorization, and $u \in \mathbb{QR}_N$. Then, the protocol $\prod_{LHTLP}$ is a public-coin honest-verifier argument corresponding to the language $\mathcal{L}_{LHTLP}$.

Proof. Completeness: Firstly, $\mathcal{P}$ proves to $\mathcal{V}$ that $w = u^{2^T} \bmod N$ using $\prod_{VDF}$. Once w is correctly verified, $\mathcal{V}$ outputs accept if $(d < N)^5$ and $(v = w^N \cdot (1 + N)^d \bmod N^2)$.

Soundness: Since N is a strong RSA integer with an unknown factorization, the low order assumption holds unconditionally in $\mathbb{QR}_N$. Given that $u \in \mathbb{QR}_N$, then $w = u^{2^T} \bmod N$ is guaranteed using $\prod_{VDF}$ with a negligible soundness error based on Theorem 1. Consequently, if $v = w^N \cdot (1 + N)^d \bmod N^2$, then $v = (u^{2^T})^N \cdot (1 + N)^d \bmod N^2$ is also guaranteed with a negligible soundness error. $\square$

Non-interactive variant. $\prod_{LHTLP}$ can be transferred to a non-interactive variant by using the non-interactive variant of $\prod_{VDF}$. Hence, the overall proof $(\pi_{LHTLP} = \pi_{VDF})$ will be $\{\mu'_i = x^{2^{(T/2^i)-1}} \bmod N\}_{i \in [t]}$.

B. Detailed phases of the e-voting protocol

We instantiate a statistically-hiding commitment scheme as:

$$\begin{aligned} \text{COMM} : \mathbb{F}_p \times \mathbb{F}_p &\rightarrow \mathbb{F}_p \\ \text{COMM}_x(\cdot) &:= H(x, \cdot) \end{aligned}$$

Setup phase: The administrator and a set of n_V eligible voters engage in two MPC-based ceremonies as follows.

1st ceremony: Let (C_V) be the zk-SNARK circuit that will be utilized during the encrypted vote casting phase. The administrator and voters generate the proving and verifying

keys (pk, vk) for C_V using the zk-SNARK's Setup algorithm with a security parameter κ : $(pk, vk) \leftarrow \text{Setup}(1^\kappa, C_V)$.

2nd ceremony: They randomly select the RSA modulus N using the method outlined in [37] s.t. $n_V < N$. Subsequently, they choose $\hat{g} \xleftarrow{\$} \mathbb{Z}_N^*$ and calculate $g := \hat{g}^2 \bmod N$ to set up g as a generator of $\mathbb{QR}_N$.

Let $T = 2^t$ represent the number of sequential squarings that encompass the time window of encrypted vote casting. Each voter calculates $h = g^{2^T} \bmod N$. Alternatively, the administrator only calculates h and then employs the VDF protocol $(\prod_{VDF})$ to prove h 's correctness. At this step, the LHTLP setup (HP.Setup) is established for the public parameter $pp = (T, N, g, h)$. They also set the block heights (T_1, T_2, T_3) to define the conclusion of the protocol phases.

Afterwards, the administrator organizes the eligible voters into a Merkle tree $\mathcal{E}$ with a tree depth $\mathfrak{D} = \lceil \log_2(n_V) \rceil$, where the addresses of the voters' blockchain accounts are the leaves. This tree and its root $root_{\mathcal{E}}$ are then publicly disclosed, enabling voters to obtain their proofs of voting eligibility $(\pi_{\mathcal{E}})$. Finally, the administrator deploys the three smart contracts (Main, IncMerkle, Verifier), then begins the voting event by invoking Main.initialize function with the protocol parameters $(T_1, T_2, T_3, vk, root_{\mathcal{E}}, n_V, N, g, h, T)$, as illustrated in Fig. 6.

Main.initialize $(T_1, T_2, T_3, vk, root_{\mathcal{E}}, n_V, N, g, h, T)$:

```

1 Store  $T_1, T_2, T_3, vk, root_{\mathcal{E}}, n_V, N, g, h, T$ 
2  $\mathcal{E}_{COM} := \text{IncMerkle.initialize}(\lceil \log_2(n_V) \rceil)$ 
3  $u := 1$  ▷ To store  $\prod u_i$ 
4  $v := 1$  ▷ To store  $\prod v_i$ 
5 regCount := 0 ▷ Current number of registered Voters
6 regVoters := mapping(addr  $\rightarrow$  {0, 1})
7 nullifiers := mapping(nullifier  $\rightarrow$  {0, 1})
```

In turn, Main.initialize (i) stores these parameters; (ii) instantiates an incremental Merkle tree $\mathcal{E}_{COM}$ with a tree depth $\mathfrak{D} = \lceil \log_2(n_V) \rceil$ using IncMerkle smart contract to store the voters' commitments (iii) initializes the variables (u, v) for accumulating the (LHTLP) encrypted votes, and regCount for tracking the number of registered voters; (iv) initializes two hash tables regVoters and nullifiers, which map a blockchain account address *addr* and a bit-string *nullifier* as hash table keys to values 0 or 1. These mappings enable efficient tracking of registered voters and revealed nullifiers during the subsequent phases.

Registration phase: Each voter V randomly selects a secret $(st \xleftarrow{\$} \mathbb{F}_p)$ and then computes a voting commitment as $cmt = \text{COMM}_{st}(addr_V)$, where $addr_V$ is the address of the voter's blockchain account. The commitment format guarantees that each *cmt* is unique even if multiple voters select the same *st*, since each $addr_V$ is unique in the blockchain. The voter also obtains the proof of voting eligibility $(\pi_{\mathcal{E}})$. Next, the voter sends Register transaction, as illustrated in Fig. 6, which invokes Main.register function with the parameters $(cmt, \pi_{\mathcal{E}})$ along with a collateral *fee* collected by the smart contract to compensate relayers in the subsequent phase.

⁵Given that $(1 + N)^d \equiv (1 + N)^{k \cdot N + d} \bmod N^2, \forall k > 0$, it is evident that $\mathcal{V}$ only accepts the value of d if it is the least positive residue of d Modulo N , which further implies that $0 \leq d < N$.

```

Main.register(cmt,  $\pi_{\mathcal{E}}$ ):
1 require( $T_{block} < T_1$ )
2 require(Verifier.merkleVerify(rootE, tx.sender,
   $\pi_{\mathcal{E}}$ ))
3 require(regVoters[tx.sender] = 0)
4 regCount := regCount + 1
5 regVoters[tx.sender] := 1
6 insIdx :=  $\mathcal{E}_{COM}$ .insert(cmt)
7 emit Register(cmt, insIdx)

```

Main.register first verifies that the transaction is received while the registration is still ongoing (i.e., $T_{block} < T_1$). Second, it ensures that the transaction sender *tx.sender* is indeed a legitimate voter by confirming the submitted proof ($\pi_{\mathcal{E}}$) along with *tx.sender* against the stored *root_E* of the voting eligibility Merkle tree using **Verifier.merkleVerify** function. To prevent double registrations from the same voter, the function also verifies that the entry corresponding to *tx.sender* in the *regVoters* hash table is 0. Up on success, the function updates the records accordingly by incrementing the *regCount* by one and storing 1 on the entry corresponding to *tx.sender* in *regVoters*. Next, it appends the submitted *cmt* to the next empty leaf in the voters' commitments Merkle tree $\mathcal{E}_{COM}$. Finally, the function emits the blockchain event **Register**(*cmt*, insIdx) to efficiently record on the blockchain that the submitted *cmt* has been inserted into the leaf with index insIdx. This event enables voters to generate their proofs of registration $\pi_{\mathcal{E}_{COM}}$ at the conclusion of the registration phase.

Encrypted vote casting phase: After ending the registration phase, each voter scans the blockchain for all events **Register**(*cmt*, insIdx) and rebuilds $\mathcal{E}_{COM}$ to generate a proof of registration $\pi_{\mathcal{E}_{COM}}$ ⁶. Additionally, the voter calculates a *nullifier* corresponding to the *cmt* as *nullifier* = **COMM_{st}**(insIdx). Since each *cmt* is unique, its insIdx is unique per $\mathcal{E}_{COM}$ and thereby guaranteeing the uniqueness of *nullifier*. The uniqueness of *cmts* and *nullifiers* prevents double voting and mimicking voters. Dispute this uniqueness, the *nullifier* and *cmt* are unlinkable when *st* is unknown, thereby enforcing the unlinkability of the voter and *nullifier*.

Let $d \in \{0, 1\}$ represent the voter's vote. The voter *V* encrypts a vote *d* as a LHTLP, $Z = (u, v)$, using **HP.Gen** algorithm, as in Fig. 2, in the following manner: first, the voter randomly selects an integer *r* from $\{1, \dots, N^2\}$, then calculates $u = g^r \bmod N$ and $v = h^{r \cdot N} \cdot (1 + N)^d \bmod N^2$.

Subsequently, the voter generates a zk-SNARK proof π_{zk} for a statement $\vec{s}$ and witness $\vec{w}$ of the arithmetic Circuit 1 (C_V). This proof validates (i) the voter's proof of membership for the commitments Merkle tree $\mathcal{E}_{COM}$, (ii) the correctness of the *nullifier*, (iii) the format of the encrypted ballot.

Circuit 1: The arithmetic circuit (C_V)

Statement $\vec{s}$: $N, g, h, (u, v), \text{root}_{\mathcal{E}_{COM}}, \text{nullifier}$
Witness $\vec{w}$: $r, d, \pi_{\mathcal{E}_{COM}}, \text{insIdx}, \text{st}, \text{addr}_V$

```

1 cmt ← COMMst(addrV)
2 Assert Merkle.verify(rootE, cmt,  $\pi_{\mathcal{E}_{COM}}$ , insIdx)
3 Assert nullifier = COMMst(insIdx)
4 Assert  $d \cdot (1 - d) = 0$  ▷ Ensure that  $d \in \{0, 1\}$ 
5 Assert  $u = g^r \bmod N$ 
6 Assert  $v = h^{r \cdot N} \cdot (1 + N)^d \bmod N^2$ 

```

Typically, the generated zk-SNARK proof π_{zk} asserts that the voter knows the witness part ($\pi_{\mathcal{E}_{COM}}, \text{insIdx}, \text{st}, \text{addr}_V$) that satisfies the circuit C_V with respect to the statement part (*root_E*, *nullifier*) as follows. Firstly, the circuit recalculates *cmt* and then verifies the provided proof of membership for that *cmt* using **Merkle.verify**, i.e., **Merkle.verify** verifies that $\pi_{\mathcal{E}_{COM}}$ is the path from the leaf *cmt* with index insIdx to the root *root_E*. Subsequently, the circuit C_V confirms the correctness of the *nullifier*. Following this, the proof π_{zk} asserts the format of the encrypted vote, i.e., the voter knows (*r*, *d*) that satisfies the circuit C_V with respect to ($N, g, h, (u, v)$). Firstly, the circuit checks $d \in \{0, 1\}$ and then confirms the correctness of the provided (*u*, *v*).

To cast the vote, the voter constructs a message containing a ballot $B = ((u, v), \text{nullifier}, \pi_{zk})$, and sends it to a trustless relayer through an anonymous network as illustrated in Fig. 6. In turn, the relayer validates the correctness of the message and constructs *CastVote* transaction. Next, the relayer forwards the transaction to the **Main** smart contract and pays the transaction fee on behalf of the voter.

```

Main.castVote( $B = ((u, v), \text{nullifier}, \pi_{zk})$ ):

```

```

1 require( $T_1 < T_{block} < T_2$ )
2 require(nullifiers[nullifier] = 0)
3 root :=  $\mathcal{E}_{COM}$ .getRoot()
4 Reconstruct  $\vec{s} := (N, g, h, (u, v), \text{root}, \text{nullifier})$ 
5 require(Verifier.zkVerify(vk,  $\vec{s}, \pi_{zk}$ ))
6 nullifiers[nullifier] := 1
7  $u := u \cdot u \bmod N$ 
8  $v := v \cdot v \bmod N^2$ 

```

Main.castVote checks whether the transaction is received during the time window of casting encrypted vote (i.e., $T_1 < T_{block} < T_2$). To prevent double voting, the function ensures that the *nullifier* has not seen before by verifying that its corresponding entry on the *nullifiers* hash table is 0. Next, it reconstructs the statement $\vec{s}$ and verifies the proof π_{zk} with respect to $\vec{s}$ using **Verifier.zkVerify**. Upon success, it marks the *nullifier* as revealed by updating the *nullifiers* hash table. Finally, it accumulates the received encrypted votes to the previously submitted ones, and compensates the relayer.

Result tallying phase: At the end of encrypted vote casting, the encrypted result of the election becomes automatically accessible on the blockchain in the form of a LHTLP. To obtain the plain result *D*, any interested party can follow these steps: first, retrieve the accumulated puzzle (*u*, *v*) from the blockchain, and then perform *T* sequential squarings on *u* to obtain $w = u^{2^T} \bmod N$, and finally, calculate *D* as $(v/w^N \bmod N^2) - 1$. Since these steps are time-consuming, they are typically performed by the administrator who utilizes the

⁶Typically, the administrator or relayers can manage to rebuild $\mathcal{E}_{COM}$ off-chain and offer it to voters.

protocol Π_{LHTLP} , introduced in the previous section, to provide evidence of the correctness of the obtained plain result.

Recalling the non-interactive variant of the protocol Π_{LHTLP} where the administrator acts as the prover $\mathcal{P}$ and Main smart contract acts as the verifier $\mathcal{V}$, Π_{LHTLP} necessitates that $(u, w) \in \mathbb{QR}_N$ and $\mu'_i \in \mathbb{Z}_N^*$ in π_{VDF} . These conditions, while essential, are not directly verifiable on the smart contract due to their computational complexity. Therefore, they are verified indirectly as follows.

Regarding the condition $u \in \mathbb{QR}_N$, its validity is assured since (i) u is computed by the smart contract as the product of the u portion of voters' ballots; (ii) each $u \in \mathbb{QR}_N$ is confirmed by the zk-SNARK proof, where the circuit C_V verifies $u = g^r \mod N$, where g is the generator of $\mathbb{QR}_N$.

For the two other conditions, the administrator calculates $w' = u^{2^{T-1}} \mod N$ and its multiplicative inverse $\bar{w}'$, rather than $w = u^{2^T} \mod N$. Subsequently, the smart contract verifies that $w' \cdot \bar{w}' \equiv 1 \mod N$, thereby confirming $w' \in \mathbb{Z}_N^*$, then by calculating $w = w'^2 \mod N$, it enforces $w \in \mathbb{QR}_N$.

Furthermore, the administrator assembles π_{VDF} as $\{\mu'_i = u^{2^{(T/2^i)-1}} \mod N\}_{i \in [t]}$ and their multiplicative inverses $\bar{\mu}'_i$. Subsequently, the smart contract validates that $\mu'_i \cdot \bar{\mu}'_i \equiv 1 \mod N$, ensuring the compliance of $\mu'_i \in \mathbb{Z}_N^*$.

Afterwards, the administrator sends a *SetTally* transaction which invokes *Main.setTally* with the parameters $(D, (w', \bar{w}'), \pi_{VDF} = \{(\mu'_i, \bar{\mu}'_i)\}_{i \in [t]})$.

Main.setTally($D, (w', \bar{w}'), \pi_{VDF}$):

```

1 require( $T_2 < T_{block} < T_3$ )
2 require(Verifier.vdfVerify( $N, u, T, (w', \bar{w}'), \pi_{VDF}$ ))
3 require( $D \leq \text{regCount} < N$ )
4 require( $((w')^{2 \cdot N} \cdot (1 + N)^D \mod N^2 = v)$ )
5 tallyingResult := D

```

Verifier.vdfVerify($N, x, T, (y', \bar{y}'), \{(\mu'_i, \bar{\mu}'_i)\}_{i \in [t]}$):

```

1 T := T, x := x
2 require( $y' \cdot \bar{y}' \mod N = 1$ )           ▷  $y' \in \mathbb{Z}_N^*$ 
3  $y := (y')^2 \mod N$                      ▷  $y \in \mathbb{QR}_N$ 
4 for  $i := 1$  to  $t$  do
5   require( $\mu'_i \cdot \bar{\mu}'_i \mod N = 1$ )       ▷  $\mu'_i \in \mathbb{Z}_N^*$ 
6    $\mu := (\mu'_i)^2 \mod N$                  ▷  $\mu \in \mathbb{QR}_N$ 
7    $T := T/2$ 
8    $\alpha := H_\lambda(T, x, y, \mu)$ 
9    $x := x^\alpha \cdot \mu \mod N$ 
10   $y := \mu^\alpha \cdot y \mod N$ 
11 end
12 require( $y = x^2 \mod N$ )

```

In response, *Main.setTally* function firstly confirms that the transaction is received after the end of the encrypted vote casting phase to guarantee that all ballots are counted. Subsequently, it verifies that the three statements of Π_{LHTLP} are true as follows. First, it verifies that the argument $(N, u, T, (w', \bar{w}'), \pi_{VDF})$ is accepted using *Verifier.vdfVerify* function which implements the non-interactive variant of Π_{VDF} . Second, since the number of 'Yes' votes ($d = 1$) cannot exceed the total number of registered voters, the function checks that $(D \leq \text{regCount} < N)$. Finally, it

calculates $w = w'^2 \mod N$ and verifies that $w^N \cdot (1 + N)^D \mod N^2$ matches the stored value of v . Upon success, it stores D on the blockchain as the official tallying result, thereby allowing anyone to access the election's outcome.

Refund phase: After setting the result, any remaining balance is distributed equally among the registered voters who can trigger this process by sending a *Refund* transaction.

C. Discussions

Below, we explore various aspects and potential extensions of the protocol.

Approval voting with multiple candidates. Consider a scenario where there are n_V voters and m candidates, and each voter has to approve some of the candidates. In this case, each voter V_j selects a vote as a vector $\vec{d}^j = [d_0^j, d_1^j, \dots, d_{m-1}^j]$, where each $d_i^j \in \{0, 1\}$. A straightforward approach would involve generating a LHTLP puzzle for each candidate, resulting in m puzzles. However, this approach has significant drawbacks: (i) Administrative overhead where the administrator would need to solve m different accumulated puzzles, one for each candidate. Consequently, the smart contract would have to verify all m solutions on-chain. This process is time-consuming and costly. (ii) Circuit complexity since the circuit C_V , during the encrypted votes casting, must validate all the puzzles simultaneously. This includes enforcing $d_i^j \in \{0, 1\}$ and ensuring different random values for each puzzle (to maintain ballot privacy). Additionally, it needs to account for any other restrictions on how candidates are elected, e.g., if each voter must approve up to $\hat{m}$ candidates, the condition $\sum_i d_i^j \leq \hat{m}$ must be verified. This increases the circuit complexity and the time of zk-SNARK proof generation.

An efficient solution is to create a single puzzle that encompasses all candidates as follows. Let $k = \lfloor \log_2 n_V \rfloor + 1$ and suppose $2^{m \cdot k} < N$. Each voter V_j aggregates the vote vector $\vec{d}^j$ into a single number d^j as

$$d^j = \sum_i d_i^j \cdot 2^{i \cdot k} = d_0^j + d_1^j \cdot 2^k + \dots + d_{m-1}^j \cdot 2^{(m-1) \cdot k}$$

Next, the voter generates a single puzzle for the aggregated vote d^j and follows the protocol in which the circuit C_V also checks the correctness of the aggregated vote and $d_i^j \in \{0, 1\}$ ⁷. Consequently, during the result tallying phase, the administrator obtains the plain D , which can be expressed as

$$\begin{aligned}
D &= \sum_j d^j = \sum_j \sum_i d_i^j \cdot 2^{i \cdot k} = \sum_i \left(\sum_j d_i^j \right) \cdot 2^{i \cdot k} \\
&= \sum_j d_0^j + \sum_j d_1^j \cdot 2^k + \dots + \sum_j d_{m-1}^j \cdot 2^{(m-1) \cdot k} \\
&= D_0 + D_1 \cdot 2^k + \dots + D_{m-1} \cdot 2^{(m-1) \cdot k}
\end{aligned}$$

In this expression, the tallying result for the i -th candidate $D_i = \sum_j d_i^j$ is encoded in a k -bit segment. Consequently, the individual tallying result can be extracted easily using $D_i = \lfloor (D \mod 2^{k \cdot (i+1)}) / 2^{i \cdot k} \rfloor$. It is worth mentioning that this approach guarantees that there is no overflow between different D_i values, as $D_i \leq n_V < 2^k$.

⁷In addition to any other restrictions on how to elect candidates.

Score voting instead of approval voting. In a scenario similar to the previous one, where there are n_V voters and m candidates, each voter V_j has the flexibility to assign a score $d_i^j \in [0, S]$ to the i -th candidate, rather than choosing between ‘Yes/No’ options. Here, S represents the predefined maximal score. In this extended scenario, we can adapt the approach used for approval voting with a new value of the parameter $k = \lceil \log_2(S \cdot n_V) \rceil + 1$, while maintaining the condition $2^{m \cdot k} < N$. Additionally, the circuit C_V is enhanced to check not only the correctness of the aggregated votes but also the range of scores assigned to candidates, $d_i^j \in [0, S]$. Any other restrictions on how scores should be assigned can also be incorporated into the circuit, for instance, the sum requirement ($\sum_i d_i^j = S$) could be easily verified by the zk-SNARK circuit.

Reuse of the protocol parameters. Ideally, it would be more efficient to establish the public parameters once and reuse them in many subsequent voting events. Recalling the Setup phase, two primary sets of parameters are involved: (T, N, g, h) for the LHTLP and (pk, vk) for the zk-SNARK circuit (C_V). As demonstrated in [23], having prior knowledge of the group structure of the LHTLP does not contribute to breaking the sequential squaring operation. Therefore, each LHTLP guarantees the confidentiality of its vote for a duration equivalent to the time required for T sequential squarings, starting from the generation of the puzzle itself rather than the generation of its public parameters. Concerning the zk-SNARK parameters, the number of constraints in C_V , and consequently its proving and verifying keys, depends on $|N|$ and $|r|$, in addition to the depth of the commitments Merkle tree $\mathcal{E}_{COM}$. Once, these variables are established, the proving and verifying keys could be reused in multiple voting events. As a result, the public parameters generated during the Setup phase could be utilized multiple times in subsequent events.

Furthermore, in cases where the voters remain the same across multiple voting events and their commitments are not revealed, the protocol could consider the registration phase as already completed and directly start from the encrypted vote casting. In such scenarios, each voting event should be assigned a unique event identifier (*eventID*), accompanied by an accumulated LHTLP (u, v) and a hash table nullifiers. To prevent replay attacks on the encrypted ballots, each *nullifier* should be associated with an *eventID* through its construction, which follows the format $nullifier = H(st, insIdx, eventID)$. The *eventID* will be included into the statement $\vec{s}$ of the zk-SNARK circuit C_V . Moreover, new voters can participate in subsequent voting events by registering their commitments. Furthermore, the protocol could be easily adapted to handle the case of revealing some voters’ commitments or excluding some voters by employing sparse Merkle trees in a revocation system.

For preserving the ballot privacy, voters should use distinct values for r for each voting event. This can be conveniently achieved by incorporating the *eventID* as a part of r .

Small group vs. large group. The current setup is suitable for a small group of voters in which each voter can participate in the Setup phase ceremonies. However, in a large group, it becomes impractical to require that all voters engage in these ceremonies. In this scenario, it would be more efficient

to delegate the Setup phase to a group of non-colluding authorities.

V. SECURITY ANALYSIS

We provide the security analysis of the protocol to demonstrate how it aligns with the design goals stated in Section III-D.

We define the property under analysis as a game between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$, where $\mathcal{A}$ wins if $\mathcal{C}$ outputs ‘1’ at the end of that game. In each game, $\mathcal{A}$ has access to the blockchain state, as well as an oracle $\mathcal{O} = \{\mathcal{O}_{Setup}, \mathcal{O}_{Reg}, \mathcal{O}_{Cast}\}$ that mimics behavior of honest parties. Practically, $\mathcal{C}$ answers $\mathcal{A}$ ’s queries to the oracle without revealing any of the trapdoors or secrets involved in producing that answers. $\mathcal{C}$ also maintains the correctness of the protocol by executing the smart contract and maintaining the blockchain state. We provide the proofs in Appendix A.

A. Anonymity

Anonymity ensures that no PPT adversary can link a given encrypted ballot to the voter who cast it. We define anonymity through a game ANON with the following steps.

Setup Step: $\mathcal{A}$ selects a set of blockchain addresses $addr_i$ ($1 \leq i \leq n_V$) and sends them to $\mathcal{O}_{Setup}$. Internally, $\mathcal{C}$ initializes an empty list L_{Reg} and executes the Setup phase.

Registration Step: $\mathcal{A}$ registers $addr_i$ by querying $\mathcal{O}_{Reg}$. Internally, $\mathcal{C}$ computes $cmt_i = \text{COMM}_{st_i}(addr_i)$ where $st_i \xleftarrow{\$} \mathbb{F}_p$ and invokes **Main.register**. Up on success, $\mathcal{C}$ stores $(addr_i, st_i)$ in L_{Reg} . $\mathcal{O}_{Reg}$ allows $\mathcal{A}$ to register a cmt_j generated by $\mathcal{A}$ for $addr_j$ by invoking **Main.register** directly.

Challenge Step: $\mathcal{A}$ selects two addresses $addr_{i_0}$ and $addr_{i_1}$ ($1 \leq i_0, i_1 \leq n_V$) as a challenge and sends them to $\mathcal{C}$ which verifies that $addr_{i_0}$ and $addr_{i_1} \in L_{Reg}$. This implies that $\mathcal{A}$ controls up to $n_V - 2$ voters.

Casting Step: $\mathcal{A}$ casts the ballot corresponding to an $addr_i$ by querying $\mathcal{O}_{Cast}$. Internally, $\mathcal{C}$ verifies that $addr_i$ is neither $addr_{i_0}$ nor $addr_{i_1}$. $\mathcal{C}$ then retrieves st_i from L_{Reg} and generates a valid $B^i = ((u^i, v^i), nullifier^i, \pi_{zk}^i)$ for a random vote d^i . Next, $\mathcal{C}$ invokes **Main.castVote**. Furthermore, $\mathcal{O}_{Cast}$ allows $\mathcal{A}$ to cast a B^j generated by $\mathcal{A}$ corresponding to an $addr_j$ by invoking **Main.castVote** directly.

Guessing Step: $\mathcal{C}$ randomly selects a bit b and responds with a ballot B^* corresponding to $addr_{i_b}$ similar to the **Casting Step**. Finally, $\mathcal{A}$ guesses b^* and $\mathcal{C}$ outputs ‘1’ if $b^* = b$.

The advantage is defined as $\text{Adv}_{\mathcal{A}}^{\text{ANON}} = |\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}|$.

Definition 1 (Anonymity). *The proposed e-voting protocol maintains the anonymity property if for any PPT adversary $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{A}}^{\text{ANON}}$ is negligible.*

Claim 1. *The proposed e-voting protocol satisfies the anonymity property under Def. 1.*

B. Time-bounded ballot privacy

Time-bounded ballot privacy is essential for preserving vote confidentiality, maintaining fairness, and preventing voting bias. To achieve time-bounded ballot privacy, the votes within

encrypted ballots must remain secret against all adversaries, even in parallel, for a predefined time period (expressed as T sequential squarings) that covers the casting vote phase.

Inspired by the security proof of LHTLP [23], we define time-bounded ballot privacy with a game TB – PRIVACY, where $\mathcal{A}$'s depth is bounded by T . TB – PRIVACY is as the game ANON in the *Setup*, *Registration*, and *Casting Steps*.

Challenge Step: $\mathcal{A}$ selects an address $addr_{i_c}$ as a challenge and sends it to $\mathcal{C}$ which verifies that $addr_{i_c} \in L_{Reg}$.

Guessing Step: $\mathcal{C}$ randomly selects a vote d and responds with a ballot B^* corresponding to $addr_{i_c}$ similar to the *Casting Step*. Finally, $\mathcal{A}$ guesses d^* and $\mathcal{C}$ outputs '1' if $d^* = d$.

The $\mathcal{A}$'s advantage is defined as $\text{Adv}_{\mathcal{A}}^{\text{TB-PRIVACY}} = |\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}|$.

Definition 2 (Time-bounded ballot privacy). *The proposed e-voting protocol maintains the time-bounded ballot privacy property if for any PPT adversary $\mathcal{A}$, whose depth is bounded by T , the advantage $\text{Adv}_{\mathcal{A}}^{\text{TB-PRIVACY}}$ is negligible.*

Claim 2. *The proposed e-voting protocol satisfies the time-bounded ballot privacy property under Def. 2.*

C. Double voting resistance

Double voting resistance is mandatory for any election system to ensure that eligible voters can vote only once.

Voters cast their ballots anonymously through trustless relayers to maintain their anonymity. Therefore, the smart contract may receive many valid ballots from the same address of one of these relayers. Accordingly, each ballot is uniquely identified by a *nullifier* since a registered voter can generate only one valid *nullifier*. Consequently, we focus on *nullifiers* rather than addresses in the definition of the double voting resistance game D – VOT as in the following steps.

Setup Step: $\mathcal{A}$ selects an address $addr_{\mathcal{A}}$ and sends it to $\mathcal{O}_{Setup}$. Internally, $\mathcal{C}$ executes the Setup phase including generating the eligibility Merkle tree $\mathcal{E}$ with a root $root_{\mathcal{E}}$.

Registration Step: $\mathcal{A}$ queries $\mathcal{O}_{Reg}$ trying to register multiple commitments cmt_j generated by himself corresponding to multiple addresses $addr_j$ (including $cmt_{\mathcal{A}}$ and $addr_{\mathcal{A}}$) by invoking `Main.register` directly.

Casting Step: $\mathcal{A}$ generates two **valid** ballots (B^0, B^1) with two different *nullifiers*, and queries $\mathcal{O}_{Cast}$ to cast these ballots by invoking `Main.castVote` directly.

Output Step: $\mathcal{C}$ outputs '1' if B^0 and B^1 are accepted by the smart contract.

The $\mathcal{A}$'s advantage is defined as $\text{Adv}_{\mathcal{A}}^{\text{D-VOT}} = \Pr[\mathcal{A} \text{ wins}]$.

Definition 3 (Double voting resistance). *The proposed e-voting protocol maintains the double voting resistance property if for any PPT adversary $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{A}}^{\text{D-VOT}}$ is negligible.*

Claim 3. *The proposed e-voting protocol satisfies the double voting resistance property under Def. 3.*

D. Universal verifiability

Universal verifiability is a critical property that allows any interested party to efficiently verify the final tally without re-tallying and ensures that no one can falsify the final tally. In the

proposed protocol, this means that the election administrator who performs the final tally can convince any interested party with the correct tallying result.

We define the universal verifiability through a game U – VERIFIABILITY with the following steps.

Setup Step: it is as the *Setup Step* of the game ANON.

Registration Step: It is as the *Registration Step* of the game ANON expect $\mathcal{A}$ cannot register a cmt_j generated by himself.

Casting Step: $\mathcal{C}$ internally instantiates a variable $D := 0$ to store the accumulated tallying result. When $\mathcal{A}$ casts the ballot corresponding to an $addr_i$ by querying $\mathcal{O}_{Cast}$, $\mathcal{C}$ retrieves st_i from L_{Reg} and generates a valid $B^i = ((u^i, v^i), nullifier^i, \pi_{zk}^i)$ for a random vote d^i . Next, $\mathcal{C}$ invokes `Main.castVote`, and updates D as $D := D + d^i$.

Guessing Step: $\mathcal{A}$ retrieves the encrypted election result in the form of an accumulated LHTLP (u, v) . Next, $\mathcal{A}$ invokes the smart contract function `Main.setTally` with the parameters $(D^*, (w', \bar{w}'), \pi_{VDF} = \{(\mu'_i, \bar{\mu}'_i)\})$. In response, $\mathcal{C}$ outputs '1' if $D^* \neq D$ and the smart contract accepts D^* as the final tally.

The $\mathcal{A}$'s advantage is defined as $\text{Adv}_{\mathcal{A}}^{\text{U-VERIFIABILITY}} = \Pr[\mathcal{A} \text{ wins}]$.

Definition 4 (Universal verifiability). *The proposed e-voting protocol maintains the universal verifiability property if for any PPT adversary $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{A}}^{\text{U-VERIFIABILITY}}$ is negligible.*

Claim 4. *The proposed e-voting protocol satisfies the universal verifiability property under Def. 4.*

E. Further analysis

Regrading the remaining properties, we argue that our proposed protocol satisfies ballot validity, individual verifiability, dispute-freeness, as well as fairness, abandon resistance, and availability.

Ballot validity. It is achieved through zk-SNARK soundness, which makes it difficult for adversaries to generate a false proof s.t. its puzzle becomes invalid and/or its *nullifier* does not belong to any of the registered voters.

Individual verifiability. It is accomplished since ballots are protected from tampering by the ballot validity (*cast as intended*), and the smart contract effectively accumulates these ballots to assure that they are accurately *recorded as cast*.

Dispute-freeness. It is guaranteed since voters provide proofs of eligibility $(\pi_{\mathcal{E}})$, and zk-SNARK proofs (π_{zk}) that validate both the voters' proofs of registration and the correctness of the ballots. These proofs are verifiable by the smart contract and any interested party.

Fairness. Partial tallying during the encrypted vote casting phase is prevented by time-bounded ballot privacy and the infeasibility of solving the accumulated LHTLP prematurely before the end of the encrypted vote casting phase.

Abandon resistance. The protocol inherently satisfies this property through its design. Each encrypted ballot is self-contained and independent of other voters, allowing the final tally to be performed efficiently with only the submitted ballots, even if some registered voters decide to not cast their ballots.

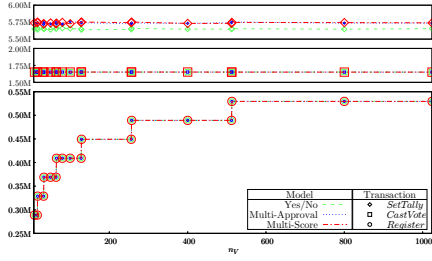


Fig. 7: Gas cost of *Register*, *CastVote*, and *SetTally* transactions, where # of Voters (n_V) varies from 9 to 1024.

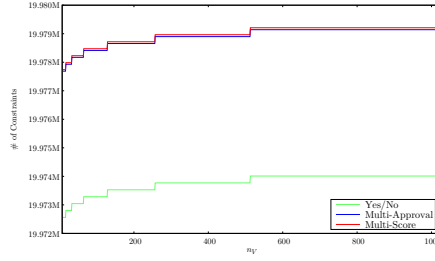


Fig. 8: # of Constraints of the zk-SNARK circuit C_V , where # of Voters (n_V) varies from 9 to 1024.

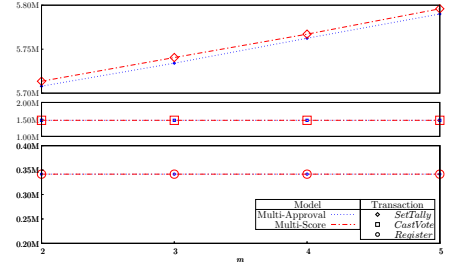


Fig. 9: Gas cost of *Register*, *CastVote*, and *SetTally* transactions, where # of Candidates (m) varies from 2 to 5.

Availability. The protocol ensures resistance against DoS attacks through blockchain decentralization. Voter anonymity and ballot privacy prevent DoS attacks by relayers and miners targeting specific voters and/or candidates. Abandon resistance mitigates DoS risks from some voters, hence preventing disruptions to the tallying process.

VI. IMPLEMENTATION AND EVALUATION

In this section, we explore the feasibility of implementing the protocol in a realistic context where N is a 1024-bit RSA integer⁸. We developed a proof-of-concept implementation of our design on the Ethereum blockchain. It is available as an open-source project on Github⁹. The details of this implementation such as justifications of the concrete values of the implementation parameters can be found in Appendix B. It is worth noting that combining sharding blockchains, such as [42], Omniledger [43], RapidChain [44], SSHC [45], and CHERUBIM [46], as Layer 2 solutions could enhance the throughput at the production level.

We utilize Groth16 zk-SNARK protocol [35] to build the zk-SNARK circuit (C_V). Poseidon hash function [47] is chosen for implementing H and the hash function for the Merkle tree $\mathcal{E}_{COM}$. Furthermore, SHA-256 is selected for implementing H_λ and the hash function for the Merkle tree $\mathcal{E}$.

Further details related to rationales of these selections, cryptographic libraries, and the hardware and environment of this implementation are provided in Appendix B.

We conducted several experiments to assess the functionality of our original design, referred to as the ‘Yes/No’ model. We also assessed the extensions of multiple candidates where each voter can approve some of them, referred to as the ‘Multi-Approval’ model, or assign scores to them, referred to as the ‘Multi-Score’ model. Furthermore, we demonstrated the performance of the proposed voting models: ‘Yes/No’, ‘Multi-Approval’, and ‘Multi-Score’ in terms of the gas consumption of the three transactions: *Register*, *CastVote*, and *SetTally*; and the number of constraints of the zk-SNARK circuit C_V . The latter is the key factor in the size of proving key (pk), hence, the zk-SNARK proof generation time. To this end,

we conducted two experiments. The first one illustrates the effect of the number of voters n_V . Specifically, the number of voters varied from 9 to 1024, and the number of candidates for ‘Multi-Approval’ and ‘Multi-Score’ is fixed at 3. Then, we compared the gas consumption of the three transactions (Fig. 7) and counted the number of constraints of the zk-SNARK circuit C_V (Fig. 8). Furthermore, the second experiment illustrates the effect of the number of candidates in ‘Multi-Approval’ and ‘Multi-Score’. Therefore, the number of voters n_V was fixed to 100 and the number of candidates varied from 2 to 5, then we compared the gas consumption of the three transactions (Fig. 9).

As depicted in Fig. 7, the gas consumption of the *Register* transaction is identical across the three models and increases logarithmically with the number of voters (n_V), i.e., linearly with the tree depth $\mathfrak{D} = \lceil \log_2(n_V) \rceil$. This behavior is expected, as the `Main.register` function involves verifying the Merkle proof of the eligibility tree $\mathcal{E}$ and inserting a new commitment (*cmt*) into the voters’ commitments Merkle tree $\mathcal{E}_{COM}$. Consequently, as the number of voters increases, the depth of these trees ($\mathfrak{D}$) grows logarithmically. In particular, the gas consumption of the *Register* transaction increases in steps, i.e., each doubling of the number of voters adds an amount equal to the execution cost of the SHA-256 hash for $\mathcal{E}$ and the Poseidon hash for $\mathcal{E}_{COM}$ on the blockchain. In contrast, the *CastVote* transaction consumes a constant amount of gas across all models, regardless of the number of voters. Similarly, the gas consumption of the *SetTally* transaction is independent of the number of voters; however, it is higher in the ‘Multi-Approval’ and ‘Multi-Score’ models compared to the ‘Yes/No’ model, due to the additional step required to extract and store the individual tally results for each candidate from the aggregated result.

Recall that the proof of registration $\pi_{\mathcal{E}_{COM}}$ is verified by the zk-SNARK circuit C_V . Since the proof size increases due to the increasing depth $\mathfrak{D}$ of the voters’ commitments Merkle tree $\mathcal{E}_{COM}$, its verification requires more constraints. Specifically, the number of constraints increases logarithmically with the number of voters (n_V) as depicted in Fig. 8. For the ‘Yes/No’ model, it increases with a step due to one more Poseidon hash. For ‘Multi-Approval’ and ‘Multi-Score’, it increases with a step due to one more Poseidon hash in addition to the constraints required to verify the aggregated vote. The latter depends on k , which also increases logarithmically with the

⁸While 1024-RSA provides only an 80-bit security level, this level is adequate for our use case as long as the best factoring attack against it takes longer than the vote casting time window (T sequential squarings). Moreover, refreshing the RSA modulus between election events through a new distributed ceremony ensures that the system remains secure over time.

⁹<https://github.com/mhgharieb/zk-Evote-HTLP>

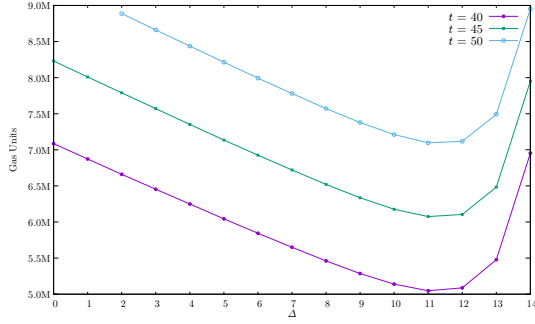


Fig. 10: The gas cost of Verifier.vdfVerify with respect to Δ .

number of voters (n_V). This is because k is calculated as $\lfloor \log_2 n_V \rfloor + 1$ in ‘Multi-Approval’ or $\lfloor \log_2(S \cdot n_V) \rfloor + 1$ in ‘Multi-Score’ as shown in Section IV-C.

As depicted in Fig. 9, the *Register* and *CastVote* transactions consistently consume gas units regardless of the number of candidates (m) for ‘Multi-Approval’ and ‘Multi-Score’. However, the gas consumption of the *SetTally* transaction increases by a fixed step with the number of candidates (m). This step arises from the process of extracting and storing the individual tallying result for each candidate from the aggregated tallying result.

Optimization Tricks. We present some optimization tricks to enhance performance. For example, the gas cost of Verifier.vdfVerify function can be reduced by executing $t - \Delta$ rounds for $\Delta > 0$, starting at $T = 2^t$ and stopping at $T = 2^\Delta$. Subsequently, it assesses the new condition $y \stackrel{?}{=} x^{2^{2^\Delta}} \bmod N$. This adjustment reduces both the VDF proof size and the associated gas cost. As demonstrated in Fig. 10, the optimal value of Δ is 11 for $t = 40, 45$, and 50. Furthermore, to mitigate the gas consumption of the Verifier.zkVerify, a semi-public statement is used in which the real statement becomes a part of the witness and the new statement is redefined to be the hash of the real one, hence the circuit verifies the correctness of this hash. Simultaneously, the smart contract reconstructs the statement by hashing the public inputs. While this approach slightly increases the number of constraints, it effectively reduces the overall gas cost.

To give concrete results, we test the implementation when the time parameter $T = 2^{40}$ associated with $\Delta = 11$. The trees depth $\mathfrak{D}$ varies between 13, 14, 15, and 16 (# of voters can reach up to $2^\mathfrak{D}$). Then, we record the number of constraints, the proving generation time, and the gas cost of the smart contract deployment as well as the gas cost of *Initialize*, *Register*, *CastVote*, and *SetTally* transactions. As shown in Table I, using semi-public statement with SHA-256 hash slightly increases the number of constraints; however, it dramatically reduces the gas cost of the *Initialize* and *CastVote* transactions.

zk-SNARK Trusted Setup Timing. Recall that the Groth16 setup [35] is divided into three major steps in two phases, namely *groth16_setup*, *contribute_per_participant*, and *final_beacon*. Using *snarkjs* library, these steps took on average 218, 42, and 41 minutes, respectively. It is worth noting that the two ceremonies in the Setup phase are or-

ganized prior to the election, so their execution time does not constitute a bottleneck to our protocol. Furthermore, the zk-SNARK setup is performed only once for an unlimited number of election events, provided that the number of voters remains unchanged (see Section IV-C).

Execution Time of Off-Chain Computation. For voters, it is dominated by zk-SNARK proof generation, whose time depends on the number of constraints in circuit C_V . Importantly, voters generate their proofs independently and in parallel. As shown in Fig. 8 and Table I, the number of constraints increases logarithmically with the number of voters (n_V) (linearly with the tree depth $\mathfrak{D}$). The incremental increase is minimal (only 242 constraints per doubling of the number of voters), therefore its impact on proof generation time is negligible. The proof generation takes less than 60 seconds on the workstation used in our experiment. While this duration may seem relatively large for an e-voting system, it can be further reduced using advanced techniques such as recursive SNARKs and folding, as we plan to explore in future work. The administrator’s off-chain computation involves solving the accumulated LHTLP to retrieve the plaintext tally and generating the proof π_{VDF} . Both tasks are independent of the number of voters and depend solely on T .

Cost of On-Chain Computation. Based on the Ether price on Feb 11, 2025¹⁰, each voter incurs a cost of approximately $\$2.46 + \mathfrak{D} \times 9.98\text{¢}$ for *Register* and *CastVote* transactions, while the administrator pays roughly $\$42.95 + \mathfrak{D} \times 19.70\text{¢}$ for the costs of the smart contract deployment, *Initialize* and *SetTally* transactions. This translates into an incremental increase of about 10¢ for each doubling of the number of voters for voters, and roughly 20¢ for the administrator. Notably, these costs can be reduced by up to 90% when deploying the smart contracts on a L2 solution such as Optimism.

VII. RELATED WORK

This section undertakes a comparative analysis of our protocol in relation to other blockchain-based self-tallying e-voting schemes. A qualitative summary is presented in Table II.

McCorry *et al.* [16] made the first attempt to implement the OVN on Ethereum. The implementation faces abortive and adaptive issues (lack of abandon resistance and fairness), as well as heavy on-chain computations that scale linearly with the number of voters. Seifelnasr *et al.* [17] partially addressed the heavy on-chain computations issue by delegating tallying computation to a trustless administrator in a verifiable manner. However, this implementation loses dispute-freeness property. ElSheikh and Youssef [18] utilized zk-SNARKs to verify off-chain computations without losing dispute-freeness property. Although they propose a variant in which the gas fees paid by voters and the administrator increase logarithmically with the number of voters, the number of zk-SNARK constraints (and thus proving times and on-chain verification costs) still grows linearly. In contrast, our approach achieves logarithmic growth in both the number of constraints and the gas fees. Li *et al.* [19] and Yang *et al.* [20] proposed ‘Yes/No’ and ‘Multi-Score’

¹⁰<https://www.cryptoneur.xyz/en/gas-fees-calculator>,
https://etherscan.io/gastracker#chart_gasprice

TABLE I: Statistics of the 1024-bit implementation including # of constraints, proving generation time, and gas costs when $T = 2^{40}$

Semi-Public Statement?	# of Constraints	Proving Time (Second)	Deploy	<i>Initialize</i>	<i>Register</i>	<i>CastVote</i>	<i>SetTally</i>
No	$19.96M + \mathfrak{D} \cdot 242$	≈ 55.559	7.873M	$5.89M + \mathfrak{D} \cdot 75k$	$79.5k + \mathfrak{D} \cdot 38k$	1.695M	5.650M
Yes using Poseidon	$19.97M + \mathfrak{D} \cdot 242$	≈ 57.322	8.168M	$3.67M + \mathfrak{D} \cdot 75k$	$79.5k + \mathfrak{D} \cdot 38k$	1.490M	5.650M
Yes using SHA-256	$21.01M + \mathfrak{D} \cdot 242$	≈ 58.981	7.880M	$2.82M + \mathfrak{D} \cdot 75k$	$79.5k + \mathfrak{D} \cdot 38k$	0.855M	5.650M

TABLE II: Comparison with related works, where $\bullet, \circ, \bigcirc$ mean the property is satisfied, partially satisfied, or not satisfied, respectively.

Property	[16]	[17]	[18]	[19]	[20]	[22]	[48]	[49]	This work
Voting Model	Yes/No	Yes/No	Yes/No	Yes/No	Score	Approval	Yes/No	Score	All
Anonymity/Voter Privacy	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$
Ballot privacy & validity	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$
Dispute-freeness	$\bullet$	$\circ$	$\bullet$	$\bullet$	$\bullet$	$\bigcirc$	$\bigcirc$	$\bullet$	$\bullet$
Fairness	$\circ$	$\circ$	$\circ$	$\bigcirc$	$\bigcirc$	$\bullet$	$\bullet$	$\bullet$	$\bullet$
Double voting resistance	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$
Individual verifiability	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$
Universal verifiability	$\bullet$	$\circ$	$\bullet$	$\bullet$	$\bullet$	$\bigcirc$	$\bigcirc$	$\bullet$	$\bullet$
Abandon resistance	$\circ$	$\circ$	$\circ$	$\bigcirc$	$\bigcirc$	$\bullet$	$\circ$	$\circ$	$\bullet$
Scalability	$\circ$	$\bigcirc$	$\bigcirc$	$\circ$	$\circ$	$\bigcirc$	$\circ$	$\bullet$	$\bullet$

e-voting models, respectively, to mitigate abortive and adaptive issues in the OVN. They add a commitment phase to the protocol. This allows recovery when a single voter refuses to cast his ballot. However, the recovery phase fails when multiple voters refuse to cast their votes. Recently, Lu *et al.* [48] addressed the fairness concerns in the OVN by incorporating time-lock puzzle and secret sharing concepts into their system. Similar to McCorry *et al.*, their implementation faces lack of abandon resistance as well as the heavy on-chain computation concerns. Furthermore, their system introduces new issues such as a lack of universal verifiability since verifying the tallying result requires re-solving the time-lock puzzle.

Li *et al.* [22] utilized LHTLP with non-interactive zero-knowledge (NIZK) proofs for binary relations to develop a ‘Multi-Approval’ self-tallying e-voting system. Despite that voters’ anonymity, ballot privacy, and fairness can be easily broken by the trusted election authority, each voter in their system creates m independent LHTLPs, a puzzle per a candidate. This approach has several drawbacks such as: (i) Solving m puzzles to derive the tallying result is time-consuming and costly, (ii) each voter creates m NIZK proofs which are expensive to verify on-chain since each verification involves several big integer modular exponentiations modulo N^2 , (iii) Creating independent puzzles prevents any criteria for candidate selection, and more importantly, threatens ballot privacy since different r for LHTLPs cannot be guaranteed. Moreover, verifying the tallying result obtained by any entity necessitates re-tallying the result, which limits universal verifiability property. Furthermore, preventing double voting requires $\binom{n_B}{2}$ on-chain comparison, where $n_B (\geq n_V)$ is the number of submitted ballots. while these comparisons are relatively inexpensive, they may lead to a linearly increasing cost of on-chain computation.

Very recently and independently of this work, Glaeser *et al.* have proposed Cicada [50], a blockchain-based e-voting framework. While voters’ anonymity is a critical property of e-voting, the Cicada framework considers user anonymity an orthogonal problem. Furthermore, it employs a non-interactive variant of Wesolowski’s VDF [30], where the verification

algorithm necessitates an *expensive* on-chain hash-to-prime function $\text{HashToPrime}(\cdot)$. While the Cicada framework does not provide a detailed description of the $\text{HashToPrime}(\cdot)$ function, its implementation on GitHub¹¹ appears to deviate from well-known approaches [51], [52] for hash-to-prime functions. Therefore, while this implementation of $\text{HashToPrime}(\cdot)$ may be less expensive, it raises concerns about the security of the implementation.

Yue proposed VeriVoting [49], a blockchain-based self-tallying, privacy-preserving weighted voting scheme. VeriVoting aims to achieve fairness and variability by distributing the required decryption key to tally the result among non-colluding candidates. However, VeriVoting lack abandon resistance when at least one candidate refuses to reveal his part of the decryption key. Moreover, VeriVoting may not be suitable for e-voting scenarios, such as the ‘Yes/No’ model, where there is only one candidate, or when candidates are considered options rather than entities capable of generating encryption/decryption keys.

Table II summarizes the comparison between the proposed protocol and previous works in terms of the security properties described in Section III-D.

VIII. CONCLUSION

In this paper, we introduced a privacy-preserving self-tallying e-voting system built on blockchain technology. The system supports various e-voting models, including ‘Yes/No’, approval voting with multiple candidates, and score voting. We leveraged Pietrzak’s verifiable delay function to propose a protocol, denoted by $(\prod_{LHTLP})$, that enables a LHTLP solver to prove the correctness of the solution without resolving the puzzle. Utilizing the LHTLP, incremental Merkle tree algorithm, and zk-SNARKs, we elaborated on the detailed phases of the e-voting protocol. The system ensure crucial properties, including voter anonymity and eligibility, as well as ballot privacy and validity. Additionally, the system achieved dispute-freeness, fairness, and abandon resistance while providing efficient universal verifiability through the protocol

¹¹<https://github.com/a16z/cicada>

($\prod_{LHTLP}$). We developed a proof-of-concept implementation of our design on the Ethereum blockchain to assess the functionality of the protocol in different e-voting models as well as the protocol's performance and feasibility. In future work, we suggest studying more desirable security properties of e-voting such as receipt-freeness and coercion resistance. Additionally, we will explore avenues for performance enhancement. This includes investigating new techniques such as recursive SNARKs and elliptic curve time-lock puzzles.

REFERENCES

- [1] D. A. Gritzalis, "Principles and requirements for a secure e-voting system," *Computers & Security*, vol. 21, no. 6, pp. 539–556, 2002.
- [2] L. Mitrou, D. Gritzalis, and S. Katsikas, "Revisiting legal and regulatory requirements for secure e-voting," *Security in the Information Society: Visions and Perspectives*, pp. 469–480, 2002.
- [3] S. Iftene, "General secret sharing based on the chinese remainder theorem with applications in e-voting," *Electronic Notes in Theoretical Computer Science*, vol. 186, pp. 67–84, 2007.
- [4] H. Pan, E. Hou, and N. Ansari, "Re-note: An e-voting scheme based on ring signature and clash attack protection," in *2013 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2013, pp. 867–871.
- [5] A. Fujioka, T. Okamoto, and K. Ohta, "A practical secret voting scheme for large scale elections," in *AUSCRYPT'92*. Springer, 1993, pp. 244–251.
- [6] D. G. Nair, V. Binu, and G. S. Kumar, "An improved e-voting scheme using secret sharing based secure multi-party computation," *arXiv preprint arXiv:1502.07469*, 2015.
- [7] P. Y. Ryan, S. Schneider, and V. Teague, "End-to-end verifiability in voting systems, from theory to practice," *IEEE Security & Privacy*, vol. 13, no. 3, pp. 59–62, 2015.
- [8] F. R. Vidal, F. Gouveia, and C. Soares, "Blockchain application in higher education diploma management and results analysis," *Advances in Science, Technology and Engineering Systems Journal*, vol. 5, no. 6, pp. 871–882, 2020.
- [9] —, "Revocation mechanisms for academic certificates stored on a blockchain," in *2020 15th Iberian Conference on Information Systems and Technologies (CISTI)*. IEEE, 2020, pp. 1–6.
- [10] F. S. Hardwick, A. Gioulis, R. N. Akram, and K. Markantonakis, "E-voting with blockchain: An e-voting protocol with decentralisation and voter privacy," in *2018 IEEE (iThings) and (GreenCom) and (CPSCoM) and (SmartData)*. IEEE, 2018, pp. 1561–1567.
- [11] A. Kiayias and M. Yung, "Self-tallying Elections and Perfect Ballot Secrecy," in *Public Key Cryptography – PKC 2002*, ser. LNCS, D. Naccache and P. Paillier, Eds., vol. 2274. Springer, 2002, pp. 141–158.
- [12] J. Groth, "Efficient Maximal Privacy in Boardroom Voting and Anonymous Broadcast," in *International Conference on Financial Cryptography – FC 2004*, ser. LNCS, vol. 3110. Springer, 2004, pp. 90–104.
- [13] F. Hao, P. Y. Ryan, and P. Zieliński, "Anonymous voting by two-round public discussion," *IET Information Security*, vol. 4, no. 2, pp. 62–67, 2010.
- [14] D. Khader, B. Smyth, P. Ryan, and F. Hao, "A fair and robust voting system by broadcast," *Lecture Notes in Informatics*, pp. 285–299, 2012.
- [15] F. Javani and A. T. Sherman, "Bvot: Self-tallying boardroom voting with oblivious transfer," *Information Security Journal: A Global Perspective*, pp. 1–12, 2022.
- [16] P. McCorry, S. F. Shahandashti, and F. Hao, "A Smart Contract for Boardroom Voting with Maximum Voter Privacy," in *International Conference on Financial Cryptography and Data Security*, ser. LNCS, vol. 10322. Springer, 2017, pp. 357–375.
- [17] M. Seifelnasr, H. S. Galal, and A. M. Youssef, "Scalable Open-Vote Network on Ethereum," in *International Conference on Financial Cryptography and Data Security*, ser. LNCS, vol. 12063. Springer, 2020, pp. 436–450.
- [18] M. ElSheikh and A. M. Youssef, "Dispute-free scalable open vote network using zk-snarks," in *Financial Cryptography and Data Security. FC 2022 International Workshops*, ser. LNCS, S. Matsuo, L. Gudgeon, A. Klages-Mundt, D. Perez Hernandez, S. Werner, T. Haines, A. Essex, A. Bracciali, and M. Sala, Eds., vol. 13412. Cham: Springer, 2023, pp. 499–515.
- [19] Y. Li, W. Susilo, G. Yang, Y. Yu, D. Liu, X. Du, and M. Guizani, "A Blockchain-Based Self-Tallying Voting Protocol in Decentralized IoT," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 1, pp. 119–130, 2022.
- [20] Y. Yang, Z. Guan, Z. Wan, J. Weng, H. H. Pang, and R. H. Deng, "Priscore: blockchain-based self-tallying election system supporting score voting," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4705–4720, 2021.
- [21] R. L. Rivest, A. Shamir, and D. A. Wagner, "Time-lock puzzles and timed-release crypto," *Massachusetts Institute of Technology*, 1996.
- [22] H. Li, Y. Li, Y. Yu, B. Wang, and K. Chen, "A Blockchain-Based Traceable Self-Tallying E-Voting Protocol in AI Era," *IEEE Transactions on Network Science and Engineering*, vol. 8, no. 2, pp. 1019–1032, 2021.
- [23] G. Malavolta and S. A. K. Thyagarajan, "Homomorphic time-lock puzzles and applications," in *Annual International Cryptology Conference*. Springer, 2019, pp. 620–649.
- [24] J. A. Halderman and V. Teague, "The new south wales ivote system: Security failures and verification flaws in a live online election," in *E-Voting and Identity: 5th International Conference, VoteID 2015, Bern, Switzerland, September 2-4, 2015, Proceedings 5*. Springer, 2015, pp. 35–53.
- [25] D. Springall, T. Finkenauer, Z. Durumeric, J. Kitcat, H. Hursti, M. MacAlpine, and J. A. Halderman, "Security analysis of the estonian internet voting system," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, 2014, pp. 703–715.
- [26] M. A. Specter, J. Koppel, and D. Weitzner, "The ballot is busted before the blockchain: A security analysis of voatz, the first internet voting application used in {US}. federal elections," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 1535–1553.
- [27] P. Gaudry and A. Golovnev, "Breaking the encryption scheme of the moscow internet voting system," in *Financial Cryptography and Data Security: 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10–14, 2020 Revised Selected Papers 24*. Springer, 2020, pp. 32–49.
- [28] D. Boneh, J. Bonneau, B. Bünz, and B. Fisch, "Verifiable delay functions," in *Annual international cryptology conference*. Springer, 2018, pp. 757–788.
- [29] K. Pietrzak, "Simple verifiable delay functions," in *10th innovations in theoretical computer science conference (itcs 2019)*. Schloss-Dagstuhl-Leibniz Zentrum für Informatik, 2019.
- [30] B. Weslowski, "Efficient verifiable delay functions," in *Advances in Cryptology—EUROCRYPT 2019: 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19–23, 2019, Proceedings, Part III 38*. Springer, 2019, pp. 379–407.
- [31] V. Attias, L. Vigneri, and V. Dimitrov, "Implementation study of two verifiable delay functions," *Cryptology ePrint Archive*, Paper 2020/332, 2020, <https://eprint.iacr.org/2020/332>.
- [32] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Conference on the theory and application of cryptographic techniques*. Springer, 1986, pp. 186–194.
- [33] D. Park, Y. Zhang, and G. Rosu, "End-to-end formal verification of ethereum 2.0 deposit smart contract," in *Computer Aided Verification*, S. K. Lahiri and C. Wang, Eds. Cham: Springer, 2020, pp. 151–164.
- [34] J. Groth, "Short Pairing-Based Non-interactive Zero-Knowledge Arguments," in *Advances in Cryptology – ASIACRYPT 2010*, ser. LNCS, vol. 6477. Springer, 2010, pp. 321–340.
- [35] —, "On the Size of Pairing-Based Non-interactive Arguments," in *Advances in Cryptology – EUROCRYPT 2016*, ser. LNCS, vol. 9666. Springer, 2016, pp. 305–326.
- [36] V. Nikolaenko, S. Ragsdale, J. Bonneau, and D. Boneh, "Powers-of-Tau to the People: Decentralizing Setup Ceremonies," in *Applied Cryptography and Network Security*, C. Pöpper and L. Batina, Eds. Cham: Springer Nature Switzerland, 2024, pp. 105–134.
- [37] M. Chen, C. Hazay, Y. Ishai, Y. Kashnikov, D. Micciancio, T. Riviere, A. Shelat, M. Venkatasubramanian, and R. Wang, "Diogenes: Lightweight scalable rsa modulus generation with a dishonest majority," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 590–607.
- [38] V. Goyal, Y. Liu, and Y. Song, "Communication-efficient unconditional mpc with guaranteed output delivery," in *Advances in Cryptology – CRYPTO 2019*, A. Boldyreva and D. Micciancio, Eds. Cham: Springer International Publishing, 2019, pp. 85–114.
- [39] V. Goyal, Y. Song, and C. Zhu, "Guaranteed output delivery comes free in honest majority mpc," in *Advances in Cryptology – CRYPTO 2020*,

- D. Micciancio and T. Ristenpart, Eds. Cham: Springer International Publishing, 2020, pp. 618–646.
- [40] S. Kremer, M. Ryan, and B. Smyth, “Election verifiability in electronic voting protocols,” in *Computer Security–ESORICS 2010: 15th European Symposium on Research in Computer Security, Athens, Greece, September 20–22, 2010. Proceedings 15*. Springer, 2010, pp. 389–404.
- [41] D. Boneh, B. Bünz, and B. Fisch, “A survey of two verifiable delay functions,” Cryptology ePrint Archive, Paper 2018/712, 2018, <https://eprint.iacr.org/2018/712>.
- [42] Y. Liu, X. Xing, H. Cheng, D. Li, Z. Guan, J. Liu, and Q. Wu, “A flexible sharding blockchain protocol based on cross-shard byzantine fault tolerance,” *IEEE Transactions on Information Forensics and Security*, 2023.
- [43] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, “Omniledger: A secure, scale-out, decentralized ledger via sharding,” in *2018 IEEE symposium on security and privacy (SP)*. IEEE, 2018, pp. 583–598.
- [44] M. Zamani, M. Movahedi, and M. Raykova, “Rapidchain: Scaling blockchain via full sharding,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 931–948.
- [45] Y. Liu, J. Liu, Q. Wu, H. Yu, Y. Hei, and Z. Zhou, “Sshc: A secure and scalable hybrid consensus protocol for sharding blockchains with a formal security framework,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 3, pp. 2070–2088, 2020.
- [46] A. Liu, Y. Liu, Q. Wu, B. Zhao, D. Li, Y. Lu, R. Lu, and W. Susilo, “Cherubim: A secure and highly parallel cross-shard consensus using quadruple pipelined two-phase commit for sharding blockchains,” *IEEE Transactions on Information Forensics and Security*, 2024.
- [47] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, “Poseidon: A New Hash Function for Zero-Knowledge Proof Systems,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 519–535.
- [48] Y. Lu, H. Li, L. Gao, J. Yu, Y. Yu, and H. Su, “Self-tallying e-voting with public traceability based on blockchain,” *Computer Standards & Interfaces*, vol. 88, p. 103795, 2024.
- [49] X. Yue, “Verivoting: A decentralized, verifiable and privacy-preserving scheme for weighted voting,” *Cryptology ePrint Archive*, 2023.
- [50] N. Glaeser, I. A. Seres, M. Zhu, and J. Bonneau, “Cicada: A framework for private non-interactive on-chain auctions and voting,” *Cryptology ePrint Archive*, 2023.
- [51] A. Ozdemir, R. Wahby, B. Whitehat, and D. Boneh, “Scaling verifiable computation using efficient set accumulators,” in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 2075–2092.
- [52] T. Groß, “Hashing to prime in zero-knowledge,” *Cryptology ePrint Archive*, Paper 2022/853, 2022, <https://eprint.iacr.org/2022/853>.
- [53] E. B. Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, “Zerocash: Decentralized Anonymous Payments from Bitcoin,” in *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, pp. 459–474.
- [54] V. Buterin, “EIP-197: Big integer modular exponentiation,” 2017. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-198>
- [55] V. Buterin and C. Reitwiessner, “EIP-197: Precompiled contracts for optimal ate pairing check on the elliptic curve alt_bn128,” 2017. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-197>
- [56] P. C. Van Oorschot and M. J. Wiener, “Parallel collision search with cryptanalytic applications,” *Journal of cryptology*, vol. 12, pp. 1–28, 1999.
- [57] S. Galbraith, J. Pollard, and R. Ruprai, “Computing discrete logarithms in an interval,” *Mathematics of Computation*, vol. 82, no. 282, pp. 1181–1195, 2013.



Muhammad ElSheikh received the B.Sc. and M.Sc. degrees from Cairo University, Cairo, Egypt, in 2007 and 2015, respectively, and the Ph.D. degree from Concordia University, Montreal, QC, Canada in 2021. ElSheikh is currently an assistant professor at the National Institute of Standards, Cairo, Egypt. He is also a postdoctoral fellow in the Department of Electrical and Computer Engineering, University of Waterloo, ON, Canada. His research interests include cryptanalysis, applied cryptography, blockchains, and ML for malware analysis.



Amr Youssef received the B.Sc. and M.Sc. degrees from Cairo University, Cairo, Egypt, in 1990 and 1993 respectively, and the Ph.D. degree from Queens University, Kingston, ON., Canada, in 1997. Dr. Youssef is currently a professor at the Concordia Institute for Information Systems Engineering (CI-ISE) at Concordia University, Montreal, Canada. His research interests include cryptology, security and privacy.



Anwar Hasan has been a faculty member at the University of Waterloo, Ontario, Canada, since in 1993 and is currently the Ripple Chair and a Professor in the Department of Electrical and Computer Engineering. From January 2013 to April 2018, he was the Faculty of Engineering’s Associate Dean of Research and External Partnerships. He is a licensed professional engineer of Ontario.

APPENDIX A
SECURITY PROOF

1) *Proof of Claim 1 (Anonymity)*

Proof. We prove Claim 1 by a series of games where $\Pr[G_i]$ is the probability of $\mathcal{A}$ winning the game G_i .

Game G_0 : It is as the original game ANON. $\Pr[G_0] = \Pr[b^* = b]$.

Game G_1 : It is the same as G_0 expect $\mathcal{C}$ executes $(pk, vk, trp) \leftarrow \text{Sim}(1^\kappa, C_V)$ rather than $(pk, vk) \leftarrow \text{Setup}(1^\kappa, C_V)$ during the *Setup Step* and computes $\pi_{zk}^i \leftarrow \text{Sim}(\vec{s}_i, trp)$, without using any witnesses, rather than $\pi_{zk}^i \leftarrow \text{Prove}(pk, \vec{s}_i, \vec{w}_i)$ during the *Guessing Step*. Therefore, $\mathcal{A}$'s advantage of distinguishing between the two games is $\mathcal{A}$'s advantage against the zero-knowledge property (ϵ_{zk}). Consequently, $|\Pr[G_0] - \Pr[G_1]| \leq \epsilon_{zk}$.

Game G_2 : It is the same as G_1 expect $\mathcal{C}$ randomly selects $nullifier^*$ from $\mathbb{F}_p$ independent of $addr_{ib}$ rather than computing it as $nullifier^* = \text{COMM}_{st_{ib}}(\text{insIdx}_{ib})$ during the *Guessing Step*. Therefore, $\mathcal{A}$'s advantage of distinguishing between the two games is $\mathcal{A}$'s advantage against the hiding property of COMM_x (ϵ_{COM}). Consequently, $|\Pr[G_1] - \Pr[G_2]| \leq \epsilon_{COM}$. Furthermore, we argue that $\mathcal{A}$'s probability of winning G_2 is $\Pr[G_2] = \frac{1}{2}$ since B^* is computed independently of the bit b .

Consequently, we argue that $\text{Adv}_{\mathcal{A}}^{\text{ANON}} \leq \epsilon$ where $\epsilon = \epsilon_{zk} + \epsilon_{COM}$ which is negligible. $\square$

2) *Proof of Claim 2 (Time-bounded ballot privacy)*

We follow the security proof of LHTLP provided in [23] to prove Claim 2. Before that, we present some intractable assumptions related to this proof in $\mathbb{QR}_N$ rather than $\mathbb{J}_N$.

Assumption 1 (Sequential Squaring). Let N be a uniform strong RSA integer, g be a generator of $\mathbb{QR}_N$, and $T(\cdot)$ be a polynomial. Then there exists some $0 < \epsilon < 1$ such that for every polynomial-size adversary $\mathcal{A} = \{\mathcal{A}_\tau\}_{\tau \in \mathbb{N}}$ who's depth is bounded from above by $T^\epsilon(\tau)$, there exists a negligible function $\epsilon_{SS}(\tau)$ such that

$$\Pr \left[b \leftarrow \mathcal{A}(N, g, T(\tau), \alpha, \beta) : \begin{array}{l} \alpha \xleftarrow{\$} \mathbb{QR}_N; b \xleftarrow{\$} \{0, 1\} \\ \text{if } b = 0 \text{ then } \beta \xleftarrow{\$} \mathbb{QR}_N \\ \text{if } b = 1 \text{ then } \beta := \alpha^{2^{T(\tau)}} \end{array} \right] \leq \frac{1}{2} + \epsilon_{SS}(\tau)$$

Assumption 2 (Decisional Diffe-Hellman). Let N be a uniform strong RSA integer, g be a generator of $\mathbb{QR}_N$. Then for every polynomial-size adversary $\mathcal{A} = \{\mathcal{A}_\tau\}_{\tau \in \mathbb{N}}$, there exists a negligible function $\epsilon_{DDH}(\tau)$ such that

$$\Pr \left[b \leftarrow \mathcal{A}(N, g, g^x, g^y, g^z) : \begin{array}{l} (x, y) \xleftarrow{\$} \{1, \dots, \phi(N)/4\}; b \xleftarrow{\$} \{0, 1\} \\ \text{if } b = 0 \text{ then } z \xleftarrow{\$} \{1, \dots, \phi(N)/4\} \\ \text{if } b = 1 \text{ then } z := x \cdot y \mod \phi(N)/4 \end{array} \right] \leq \frac{1}{2} + \epsilon_{DDH}(\tau)$$

Assumption 3 (Decisional Composite Residuosity). Let N be a uniform strong RSA integer. Then for every polynomial-size adversary $\mathcal{A} = \{\mathcal{A}_\tau\}_{\tau \in \mathbb{N}}$, there exists a negligible function $\epsilon_{DCR}(\tau)$ such that

$$\Pr \left[b \leftarrow \mathcal{A}(N, \beta) : \begin{array}{l} \alpha \xleftarrow{\$} \mathbb{Z}_N^*; b \xleftarrow{\$} \{0, 1\} \\ \text{if } b = 0 \text{ then } \beta \xleftarrow{\$} \mathbb{Z}_{N^2}^* \\ \text{if } b = 1 \text{ then } \beta := \alpha^N \end{array} \right] \leq \frac{1}{2} + \epsilon_{DCR}(\tau)$$

Proof. We prove Claim 2 by a series of games similar to Claim 1.

Game G_0 : It is as the original game TB-PRIVACY. $\Pr[G_0] = \Pr[d^* = d]$.

Game G_1 : It is the same as G_0 expect $\mathcal{C}$ executes $(pk, vk, trp) \leftarrow \text{Sim}(1^\kappa, C_V)$ rather than $(pk, vk) \leftarrow \text{Setup}(1^\kappa, C_V)$ during the *Setup Step* and computes $\pi_{zk}^i \leftarrow \text{Sim}(pk, \vec{s}_i, trp)$, without using any witnesses, rather than $\pi_{zk}^i \leftarrow \text{Prove}(pk, \vec{s}_i, \vec{w}_i)$ during the *Guessing Step*. Therefore, $\mathcal{A}$'s advantage of distinguishing between the two games is $\mathcal{A}$'s advantage against the zero-knowledge property (ϵ_{zk}). Consequently, $|\Pr[G_0] - \Pr[G_1]| \leq \epsilon_{zk}$.

Game G_2 : It is the same as G_1 expect $\mathcal{C}$ uniformly selects h from $\mathbb{QR}_N$ (written as $h = g^x$ for some random $x \in \{1, \dots, \phi(N)/4\}$) rather than computing it as $h = g^{2^T} \mod N$ during the *Guessing Step*. This h is used to compute $v^* = h^{r \cdot N} \cdot (1 + N)^d \mod N^2$. Therefore, $\mathcal{A}$'s advantage of distinguishing between the two games is $\mathcal{A}$'s advantage against the sequential squaring assumption (ϵ_{SS}). Consequently, $|\Pr[G_1] - \Pr[G_2]| \leq \epsilon_{SS}$.

Game G_3 : It is the same as G_2 expect $\mathcal{C}$ uniformly selects r from $\{1, \dots, \phi(N)/4\}$ rather than $\{1, \dots, N^2\}$ during the *Guessing Step*. This r is used to compute $v^* = (h^r)^N \cdot (1 + N)^d \mod N^2$. Based on [23, Lemma 1], the two games are statistically indistinguishable. Therefore, $\mathcal{A}$'s advantage of distinguishing between the two games is $\mathcal{A}$'s advantage against [23, Lemma 1] (ϵ_{Lemma1}). Consequently, $|\Pr[G_1] - \Pr[G_2]| \leq \epsilon_{\text{Lemma1}}$.

Game G_4 : It is the same as G_3 expect $\mathcal{C}$ uniformly selects u^* from $\mathbb{QR}_N$ (written as $u^* = g^y$ for some random $y \in \{1, \dots, \phi(N)/4\}$) rather than computing it as $u^* = g^r \mod N$ during the *Guessing Step*. Therefore, $\mathcal{A}$'s advantage of distinguishing between the two games is $\mathcal{A}$'s advantage against the DDH assumption (ϵ_{DDH}). In particular, $\mathcal{C}$ can build a distinguisher against DDH assumption on input a uniform tuple (N, g, g^x, g^y, g^z) in $\mathbb{QR}_N$ as follows. $\mathcal{C}$ instantiates $h = g^x, u^* = g^y$ and $h^r = g^z$ for some random $r \in \{1, \dots, \phi(N)/4\}$. Next, $\mathcal{C}$ feeds $\mathcal{A}$ with $(u^*, v^*) = (g^y, (g^z)^N \cdot (1 + N)^d)$ that are distributed identically to G_4 . When the tuple is a DDH tuple, $g^z = g^{x \cdot y}$ and $h^r = g^z = h^y$, and (u^*, v^*) is rewritten as $(g^y, (h^y)^N \cdot (1 + N)^d)$ that are distributed identically to G_3 . Therefore, if $\mathcal{A}$ distinguishes between the two games, $\mathcal{C}$'s distinguisher can break the DDH assumption. Consequently, $|\Pr[G_3] - \Pr[G_4]| \leq \epsilon_{DDH}$.

Game G_5 : It is the same as G_4 expect $\mathcal{C}$ computes $v^* = w \cdot (1 + N)^d \mod N^2$ where w is uniformly selected from $\mathbb{Z}_{N^2}^*$ rather than computed as $w = (h^r)^N \mod N^2$ (for some random h^r as in G_4) during the *Guessing Step*. Therefore, $\mathcal{A}$'s advantage of distinguishing between the two games is $\mathcal{A}$'s advantage against the DCR assumption (ϵ_{DCR}). In particular, $\mathcal{C}$ can build a distinguisher against DCR assumption on input a tuple (N, β) as follows. $\mathcal{C}$ uniformly selects β s.t. β and $\hat{\beta}$ have the same Jacobi symbol. Next, $\mathcal{C}$ feeds $\mathcal{A}$ with $(u^*, v^*) = (u^*, w \cdot (1 + N)^d)$ where u^* is uniformly selected as in G_4 , and $w = \beta \cdot (\hat{\beta})^N$. If β is uniform in $\mathbb{Z}_{N^2}^*$, then w is also uniform and distributed identically to G_5 . On the other hand, if β is an N -th residue s.t. $\beta = \alpha^N$, then w is also N -th residue and can be expressed as $w = (\alpha \cdot \hat{\beta})^N = (h^r)^N$.

for some random h^r which is distributed identically to G_4 . Therefore, if $\mathcal{A}$ distinguishes between the two games, $\mathcal{C}$'s distinguisher can break the DCR assumption. Consequently, $|\Pr[G_4] - \Pr[G_5]| \leq \epsilon_{DCR}$.

Furthermore, we argue that $\mathcal{A}$'s probability of winning G_5 is $\Pr[G_5] = \frac{1}{2}$ since (u^*, v^*) is randomly selected in $\mathbb{QR}_N \times \mathbb{Z}_{N^2}^*$ in light of random w and d .

Consequently, we argue that $\text{Adv}_{\mathcal{A}}^{\text{TB-PRIVACY}} \leq \epsilon$ where $\epsilon = \epsilon_{zk} + \epsilon_{SS} + \epsilon_{\text{Lemma}_1} + \epsilon_{DDH} + \epsilon_{DCR}$ which is negligible. $\square$

3) Proof of Claim 3 (Double voting resistance)

Proof. Let addr_o be a blockchain address other than $\text{addr}_{\mathcal{A}}$ that is used in the *Setup Step*. Define an event E_{Reg} in which $\mathcal{A}$ was able to register a commitment cmt_o for addr_o , and $\epsilon := \Pr[E_{\text{Reg}}]$. We can decompose the event in which $\mathcal{A}$ wins the game D – VOT into the following two disjoint events:

- 1) E_w : $\mathcal{A}$ wins, and $\mathcal{A}$ was able to register addr_o .
- 2) $E_{w/}$: $\mathcal{A}$ wins, and $\mathcal{A}$ was not able to register addr_o .

Clearly, $\Pr[\mathcal{A} \text{ wins}] = \Pr[E_w] + \Pr[E_{w/}]$. Hence, to show that $\text{Adv}_{\mathcal{A}}^{\text{D-VOT}} = \Pr[\mathcal{A} \text{ wins}]$ is negligible, it suffices to argue that each of these two probabilities is negligible.

Bounding the probability of E_w :

$\Pr[E_w] = \Pr[\mathcal{A} \text{ wins} | E_{\text{Reg}}] \cdot \Pr[E_{\text{Reg}}] = 1 \cdot \epsilon$; $\Pr[\mathcal{A} \text{ wins} | E_{\text{Reg}}]$ is equal to 1 since $\mathcal{A}$ can generate the two valid ballots; one for each registered addresses $\text{addr}_{\mathcal{A}}$ and addr_o . Assume, by way of contradiction, that ϵ is non-negligible, it implies that $\mathcal{A}$ finds a Merkle proof for addr_o in the eligibility Merkle tree $\mathcal{E}$. However, $\mathcal{E}$ contains only one leaf $\text{addr}_{\mathcal{A}}$, i.e., $\mathcal{A}$ finds a collision in the hash function used in $\mathcal{E}$. This violates the collision resistance of this hash function. Consequently, ϵ , thus $\Pr[E_w]$, is negligible.

Bounding the probability of $E_{w/}$:

$\Pr[E_{w/}] = \Pr[\mathcal{A} \text{ wins} | \bar{E}_{\text{Reg}}] \cdot \Pr[\bar{E}_{\text{Reg}}] = \Pr[\mathcal{A} \text{ wins} | \bar{E}_{\text{Reg}}] \cdot (1 - \epsilon)$. Suppose by way of contradiction that $\Pr[\mathcal{A} \text{ wins} | \bar{E}_{\text{Reg}}]$ is non-negligible. Hence, $\mathcal{A}$ generates the ballots B^0 and B^1 s.t. $\text{nullifier}^0 \neq \text{nullifier}^1$ corresponding to the single commitment $\text{cmt}_{\mathcal{A}}$. In light of the deterministic commitment scheme COMM_x , this implies that $(st_0, \text{insIdx}_0) \neq (st_1, \text{insIdx}_1)$. Let EXT be the zk-SNARK extractor for $\mathcal{A}$, the witnesses $\vec{w}_i = (r_i, d^i, \pi_{\mathcal{E}_{COM}}^i, \text{insIdx}_i, st_i, \text{addr}_i)$, for $i \in \{0, 1\}$, can be obtained using EXT corresponding to the ballots B^0 and B^1 . Observe that the validity of the two witnesses $\vec{w}_i$ implies that:

- If $st_0 \neq st_1$ and $\text{insIdx}_0 = \text{insIdx}_1$ (hence $\pi_{\mathcal{E}_{COM}}^0 = \pi_{\mathcal{E}_{COM}}^1$), $\mathcal{A}$ finds two distinct openings for the commitment $\text{cmt}_{\mathcal{A}}$. This violates the binding property of the commitment scheme COMM_x .
- If $\text{insIdx}_0 \neq \text{insIdx}_1$ (hence $\pi_{\mathcal{E}_{COM}}^0 \neq \pi_{\mathcal{E}_{COM}}^1$), $\mathcal{A}$ are able to find two valid Merkle proofs in $\mathcal{E}_{COM}$ that contains only one leaf $\text{cmt}_{\mathcal{A}}$. In other words, $\mathcal{A}$ finds a collision in the hash function used in $\mathcal{E}_{COM}$. This violates the collision resistance of this hash function.

Overall, we conclude that $\Pr[\mathcal{A} \text{ wins} | \bar{E}_{\text{Reg}}]$, hence $\Pr[E_{w/}]$ is negligible. $\square$

4) Proof of Claim 4 (Universal verifiability)

Proof. Recalling that Main.setTally function implements the protocol $\prod_{\text{LHTLP}}$, by Theorems 2 and 1, $\Pr[\mathcal{A} \text{ wins}] = \Pr[\mathcal{A} \text{ breaks the soundness property of } \prod_{\text{LHTLP}}] = \Pr[\mathcal{A} \text{ breaks the soundness property of Pietrzak's VDF } (\prod_{\text{VDF}})] = \epsilon$. Assume, by way of contradiction, that ϵ is non-negligible and $\mathcal{A}$ deceives the smart contract to accept the incorrect result, this implies that $\mathcal{A}$, as a prover, manages to convince the smart contract, as a verifier, of a wrong argument (the solution of the accumulated LHTLP). In particular, $\mathcal{C}$ can build an algorithm $\mathcal{B}$ (as described in [41]) to break the low order assumption for $\mathbb{QR}_N$ with advantage at least $\epsilon' = (\epsilon/t)^2 - (\epsilon/2^\zeta)$, where ζ is the security parameter of $\mathbb{QR}_N$. Since ϵ' is negligible for $\mathbb{QR}_N$ then so must be ϵ . $\square$

APPENDIX B

IMPLEMENTATION DETAILS

Cryptographic Primitives. In constructing the zk-SNARK circuit (C_V), we employ the Groth16 zk-SNARK protocol [35]. Groth16 protocol is known for its efficiency in terms of proof size and verification cost. It operates as a quadratic arithmetic program (QAP) and offers a linear-time Setup, quasilinear-time Prove, and linear-time Verify [53].

For cryptographic hash functions, our protocol employs four hash functions: (i) H that is used for creating commitments (cmt) and nullifiers (nullifier), (ii) H_λ that is utilized within the non-interactive variant of the protocol $\prod_{\text{VDF}}$, and (iii) two distinct hash functions for the voting eligibility Merkle tree ($\mathcal{E}$) and the voters' commitments Merkle tree ($\mathcal{E}_{COM}$). To optimize the efficiency and gas costs, we evaluated two options for implementing these hash functions: Poseidon [47] and SHA-256. Poseidon is an arithmetic circuit friendly, which yields a much lower number of constraints when compared to SHA-256. However, SHA-256 consumes less gas when executed in Ethereum smart contract. Therefore, Poseidon is chosen for implementing H and the hash function for the Merkle tree $\mathcal{E}_{COM}$, since the cmnts , nullifiers , and the proof $\pi_{\mathcal{E}_{COM}}$ are verified by the zk-SNARK circuit C_V . Furthermore, SHA-256 is selected for implementing H_λ and the hash function for the Merkle tree $\mathcal{E}$, since the proofs π_{VDF} and $\pi_{\mathcal{E}}$ are verified by the smart contract.

Cryptographic Libraries. We utilize circom (v2.0) library¹² to compile the arithmetic circuit C_V . Furthermore, circomlib (2.0.2) library¹³ helps to build the circuit. During the protocol execution, we use snarkjs (v0.4.22) library¹⁴ to handle the MPC-based setup ceremony for generating the proving and verifying keys. Also, we use rapidsnark¹⁵ to generate the zk-SNARK proofs.

Hardware and Environment. Our experiments were conducted on standard hardware, specifically, a PC running Ubuntu (version 22.04). The PC is with an Intel i5-11400 CPU, featuring six cores and a clock frequency that can reach

¹²<https://docs.circom.io/>

¹³<https://github.com/iden3/circomlib>

¹⁴<https://github.com/iden3/snarkjs>

¹⁵<https://github.com/iden3/rapidsnark>

up to 2.6 GHz. It also boasts 80 GB of RAM. To facilitate the compilation and execution of the proof-of-concept implementations, we installed Node.js (v16.20.2) libraries as prerequisites. For the development of the smart contracts, we utilized Solidity (v0.8.0). To deploy and test these smart contracts, we employed the Hardhat framework, leveraging a local in-memory Ethereum blockchain environment with a block gas limit configured to a maximum of 30M gas units.

1024-bit Implementation. To implement big integer modular exponentiation within the `Verifier.vdfVerify` function of the smart contract, we utilize the `BigNumber` library¹⁶, which uses EIP-198 [54] as a building block to perform various cryptographic operations with arbitrary length over Ethereum blockchain.

The validation of the zk-SNARK proofs on Ethereum is performed over two cyclic groups with a prime order p on the elliptic curve `alt_bn128` [55]. Consequently, all operations in the zk-SNARK circuit (C_V) are preformed over $\mathbb{F}_p$. The primary operations in the zk-SNARK circuit (C_V) involve performing big integer exponentiation operations Modulo N and Modulo N^2 . Therefore, we employ the Square-and-Multiply method with the `bigint` library¹⁷ to overcome the effect of $\mathbb{F}_p$. Basically, a `BigInt` number A in the `bigint` library is represented as an array of k_R registers with n_R bits per register, denoted as $A = (a_0, a_1, \dots, a_{k_R-1}) = a_0 + a_1 \cdot 2^{n_R} + \dots + a_{k_R-1} \cdot 2^{n_R \cdot (k_R-1)}$. By enforcing $n_R < |p|/2$ and employing optimization tricks from xJSnark, big integer multiplication operations can be performed correctly in zk-SNARK circuits.

In the following, we give some concrete values of the implementation parameters.

BigInt register size. $|N| = 1024$ bits, to reduce the number of registers and constraints, we set $n_R = 114$. Thereby N, g, h and u are each represented in 9 registers, while N^2 and v are represented in 18 registers.

The LHTLP time parameter T . It represents the number of squarings covering the time window for casting encrypted votes. The current FPGA record for a single squaring Modulo a 1024-bit integer with unknown factorization is $25ns$ ¹⁸ achieved during the VDF Alliance FPGA Competition¹⁹. Thus, for example, $T = 2^{38}$ is suitable to cover the time window for more than an hour and less than two hours, and $T = 2^{51}$ covers more than a year.

The size of r in the LHTLP. It controls the number of the Square-and-Multiply operations within the zk-SNARK circuit. The higher the size of r , the higher the number of constraints, hence the higher the proving key size and the proof generation time. Typically, r is randomly selected from $\{1, \dots, N^2\}$. Therefore, $|r|$ in our case is 2048 bits. Unfortunately, this leads to a huge circuit of $\approx 160M$ constraints. Keeping r secret is vital to maintain the time-bounded ballot privacy. The adversary should not be able to obtain the correct r by solving the discrete log of u part of the puzzle in time less than T squarings over $\mathbb{QR}_N$ as shown in Section V-B. Suppose r is

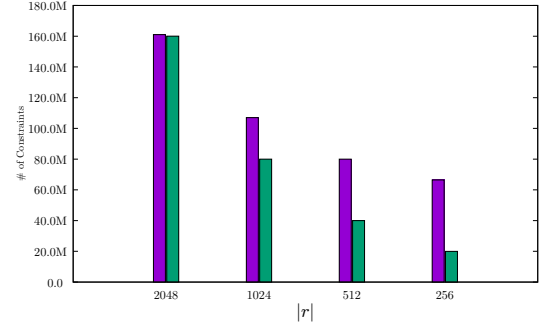


Fig. 11: The number of constraints of the zk-SNARK circuit (C_V), when $|r|$ varies in 2048, 1024, 512, and 256 bits.

selected randomly from $[1, R]$, where $R \ll \text{ord}(\mathbb{QR}_N) = p'q'$. In this case, the time-bounded ballot privacy property relies on the short-exponent discrete log assumption instead of the standard discrete log assumption. Concretely, obtaining r requires $\approx 2\sqrt{R}$ group operations using the method described in [56, Sec. 5.1] or $\approx 1.715\sqrt{R}$ group operations as in [57]. With M parallelism, it requires $\approx 2\sqrt{R}/M$ group operations using Pollard's lambda-method as described in [56, Sec. 5.1]. Therefore, the time-bounded ballot privacy is maintained as long as $T \ll \sqrt{R}$. In light of the current values of T , we select the range of r to be $[1, 2^{256}]$, i.e., $|r| = 256$ bits. Fig. 11 shows the reduction in the number of constraints when $|r|$ varies in 2048, 1024, 512, and 256 bits.

In following, we present some optimization tricks to enhance performance.

zk-SNARK circuit constraints. Recall that the v part of the puzzle can be expressed as $v = v' \cdot (1 + N)^d \mod N^2$, where $v' = h^{r \cdot N} \mod N^2$, and v' could be calculated as $v' = (h^r \mod N)^N \equiv (h^N \mod N^2)^r \mod N^2$. In other words, it could be calculated by (i) rising h to power r modulo N , then to power N modulo N^2 , or (ii) rising h to power N modulo N^2 , then to power r Modulo N^2 . Since r is a part of the witness, the circuit must verify all the computation in the first method. However, in the second method, $h^N \mod N^2$ could be publicly calculated and then passed to the circuit as part of the public statement instead of h . As depicted in Fig. 11, the total number of constraints is dramatically decreased when using the second method when $|r|$ is decreased.

The second portion of v is given by $(1 + N)^d \mod N^2$. It is easy to prove that $(1 + N)^d \equiv 1 + N \cdot d \mod N^2$. Consequently, this value can be computed using modular multiplication instead of modular exponentiation. Furthermore, given the pattern of d in the three election models, the modular value $1 + N \cdot d \mod N^2$ is equal to the absolute value $1 + N \cdot d$.

The gas cost of `Verifier.vdfVerify` function: The current implementation of the function executes the halving subprotocol for t rounds, starting at $T = 2^t$ and stopping at $T = 1$. It then assesses the condition $y = x^2 \mod N$. Alternatively, the function could execute $t - \Delta$ rounds for $\Delta > 0$, starting at $T = 2^t$ and stopping at $T = 2^\Delta$. Subsequently, it assesses the new condition $y \stackrel{?}{=} x^{2^{2^\Delta}} \mod N$. This adjustment reduces both the VDF proof size and the associated gas cost. As demonstrated in Fig. 10, the optimal value of Δ is 11 for

¹⁶<https://github.com/firoorg/solidity-BigNumber>

¹⁷<https://github.com/0xPARC/circom-cdsa/blob/master/circuits/>

¹⁸<https://github.com/supranational/vdf-fpga-round2-results>

¹⁹www.vdfalliance.org/contest

$t = 40, 45$, and 50 .

Semi-Public Statement: The gas consumption of the `Verifier.zkVerify` function, which verifies a Groth16-based zk-SNARK proof, increases linearly with the size of the statement, *i.e.*, the number of public inputs to the zk-SNARK circuit. To mitigate this cost, we redefine the statement to be the hash of these inputs. Consequently, these inputs are passed to the circuit as a part of the witness, and the circuit verifies the correctness of the hash. Simultaneously, the smart contract can reconstruct the statement by hashing the public inputs. While this approach slightly increases the number of constraints, it effectively reduces the overall gas cost.